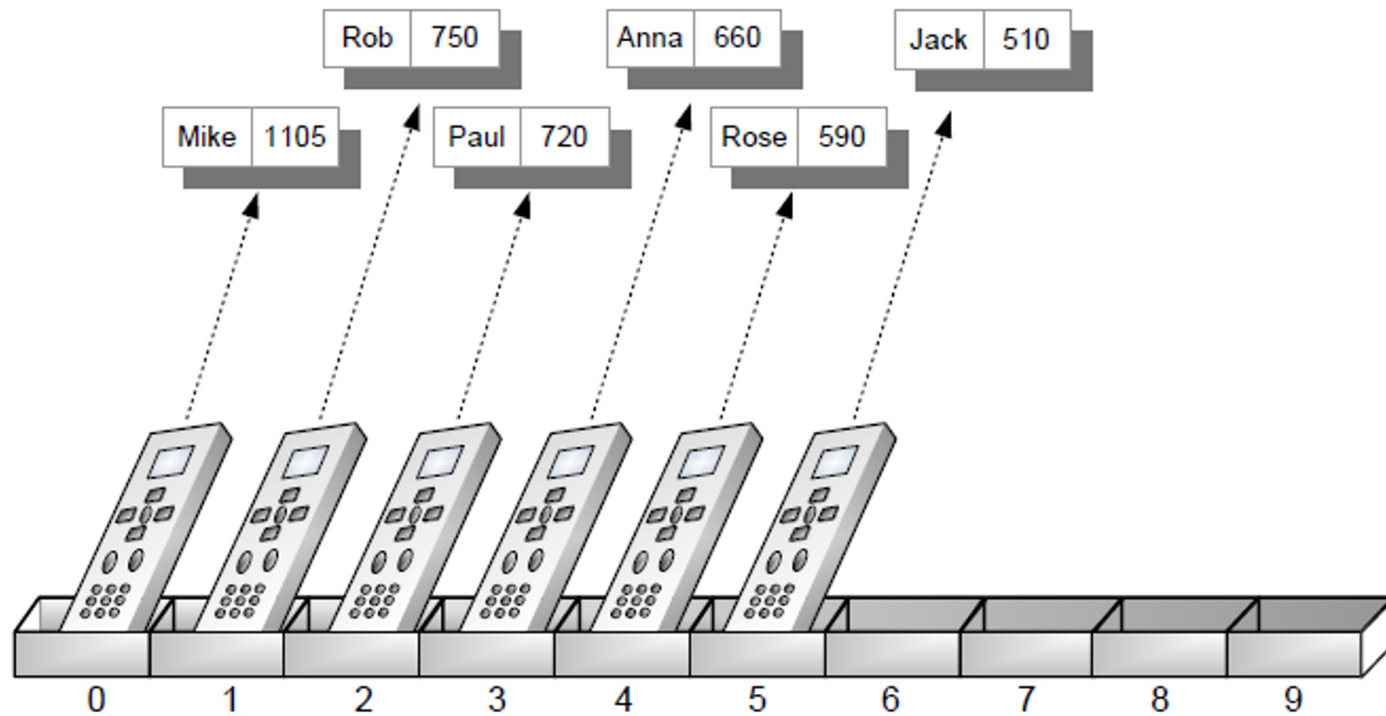


Bağlantılı Listeler (Linked List)



Diziler

Storing game entries in an array



High Score Entry

```
public class GameEntry {  
    private String name;    // name of the person earning this score  
    private int score;      // the score value  
  
    /** Constructs a game entry with given parameters.. */  
    public GameEntry(String n, int s) { name = n; score = s; }  
    /** Returns the name field. */  
    public String getName() { return name; }  
    /** Returns the score field. */  
    public int getScore() { return score; }  
    /** Returns a string representation of this entry. */  
    public String toString() { return "(" + name + ", " + score + ")"; }  
}
```

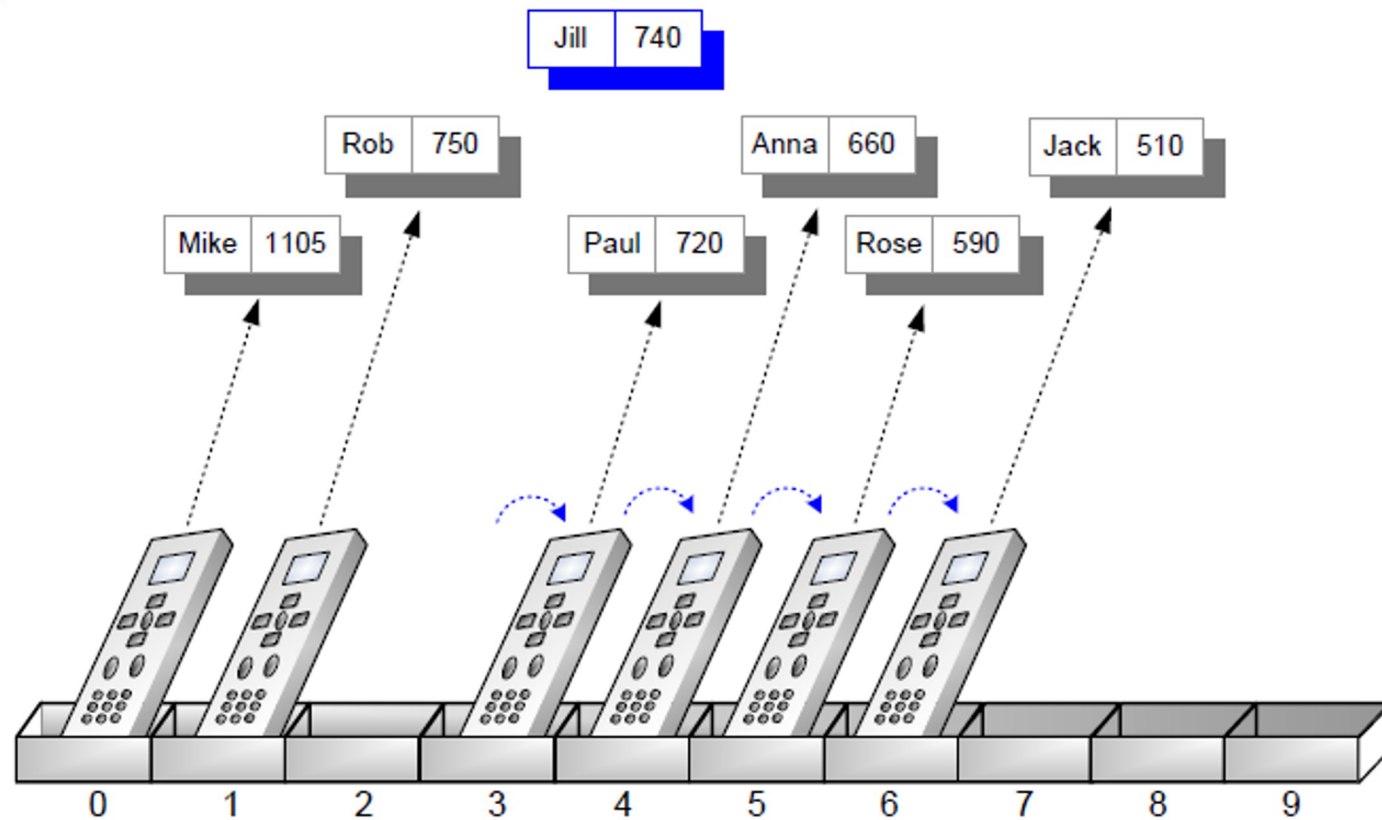
Board of high scores

```
/** Class for storing high scores in an array in nondecreasing order. */
public class Scoreboard {
    private int numEntries = 0;           // number of actual entries
    private GameEntry[] board;           // array of game entries

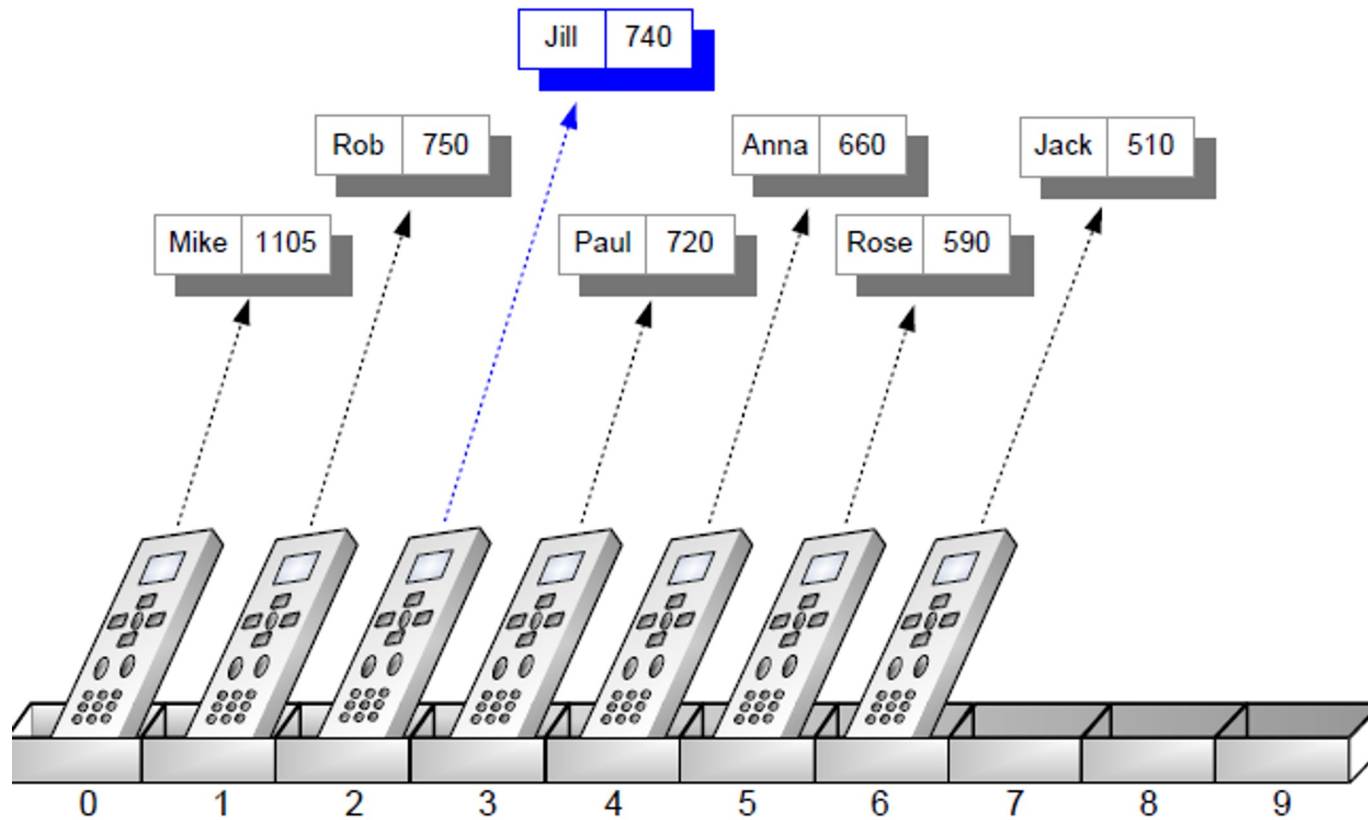
    /** Constructs an empty scoreboard with the given capacity. */
    public Scoreboard(int capacity) { board = new GameEntry[capacity]; }
    /** Attempt to add a new high score to the collection. */
    public void add(GameEntry e) {...}
    /** Remove and return the high score at index i. */
    public GameEntry remove(int i) throws IndexOutOfBoundsException {...}
    /** Returns a string representation of the high scores list. */
    public String toString() {...}

    public static void main(String[] args) {...}
}
```

Add an entry

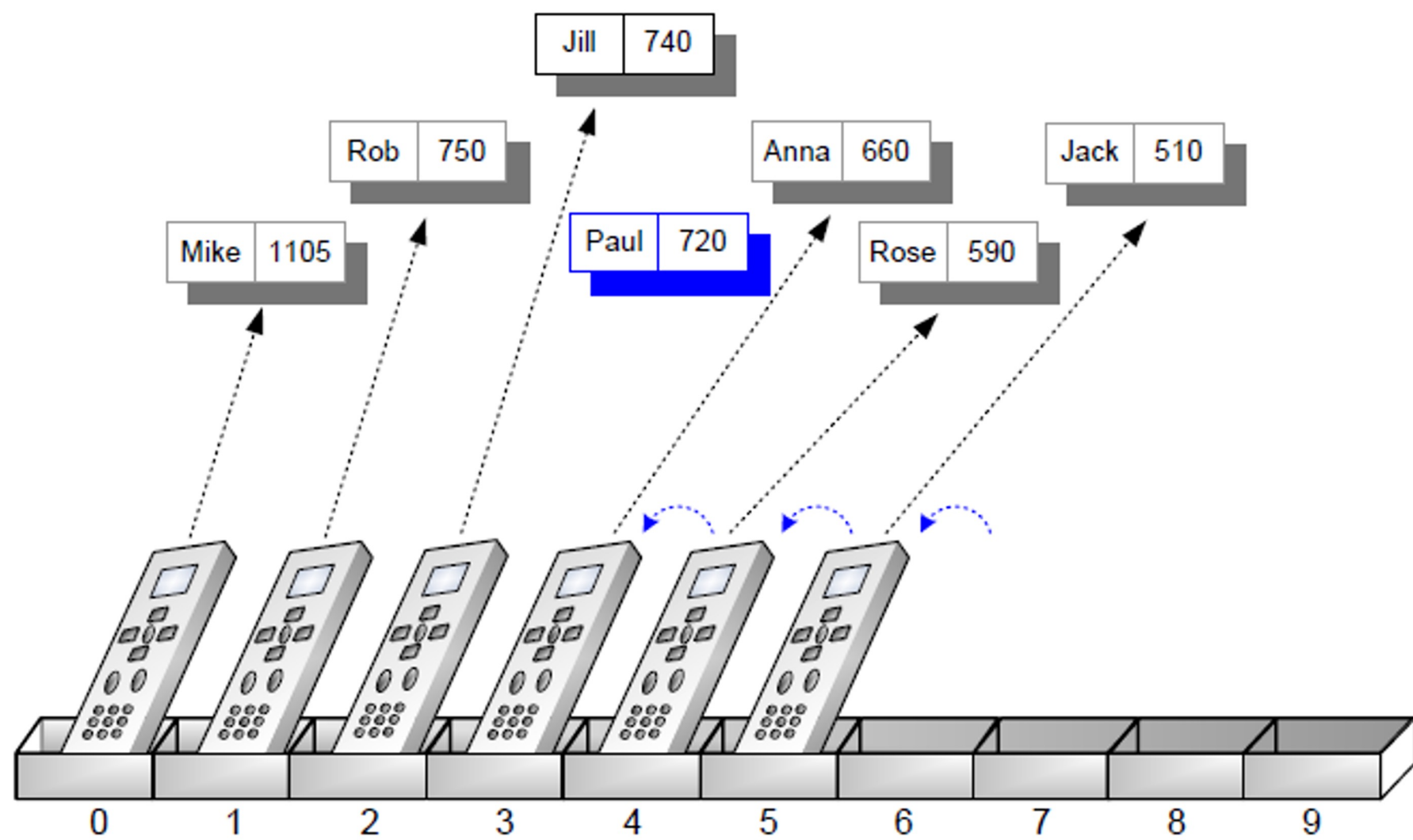


Add an entry



Add an entry

```
/** Attempt to add a new score to the collection */  
public void add(GameEntry e) {  
    int newScore = e.getScore();  
  
    // is the new entry e really a high score?  
    if (numEntries < board.length || newScore > board[numEntries-1].getScore()) {  
        if (numEntries < board.length)           // no score drops from the board  
            numEntries++;                          // so overall number increases  
        // shift any lower scores rightward to make room for the new entry  
        int j = numEntries - 1;  
        while (j > 0 && board[j-1].getScore() < newScore) {  
            board[j] = board[j-1];                // shift entry from j-1 to j  
            j--;                                    // and decrement j  
        }  
        board[j] = e;                             // when done, add new entry  
    }  
}
```



toString function

Print the board consisting of high scores:

[board[0], board[1], ..., board[numEntries-1]]

```
/** Returns a string representation of the high scores list. */
public String toString() {
    StringBuilder sb = new StringBuilder("[");

    for (int j = 0; j < numEntries; j++) {
        if (j > 0) sb.append(", ");           // separate entries by commas
        sb.append(board[j]);
    }
    sb.append("]");
    return sb.toString();
}
```

```
public static void main(String[] args) {
    Scoreboard highscores = new Scoreboard(5);
    String[] names = {"Rob","Mike","Rose","Jill","Jack","Anna","Paul","Bob"};
    int[] scores = {750, 1105, 590, 740, 510, 660, 720, 400};

    for (int i = 0; i < names.length; i++) {
        GameEntry ge = new GameEntry(names[i], scores[i]);
        System.out.println("Adding " + ge);
        highscores.add(ge);
        System.out.println(" Scoreboard: " + highscores);
    }
    System.out.println("Remove score at index " + 3); highscores.remove(3);
    System.out.println(highscores);
    System.out.println("Remove score at index " + 0); highscores.remove(0);
    System.out.println(highscores);
    System.out.println("Remove score at index " + 1); highscores.remove(1);
    System.out.println(highscores);
    System.out.println("Remove score at index " + 1); highscores.remove(1);
    System.out.println(highscores);
    System.out.println("Remove score at index " + 0); highscores.remove(0);
    System.out.println(highscores);
}
```

Built-in methods for *java.util.Arrays* class

Method	Functionality
<code>equals(A, B)</code>	Compares arrays A and B .
<code>fill(A, x)</code>	Stores x in every cell of array A .
<code>copyOf(A, n)</code>	Returns n -sized array where the first $k = \min\{n, A.length\}$ elements are copied from A . If $n > A.length$, then the remaining elements are padded with 0 or null.
<code>toString(A)</code>	Returns string representation of array A .
<code>sort(A)</code>	Sorts array A based on natural ordering.
<code>binarySearch(A)</code>	Searches the sorted array A for value x .

Advantages and disadvantages

Operation	Time complexity
Fast operations	
Access/modify	$\mathcal{O}(1)$
Insert last	$\mathcal{O}(1)$
Slow operations	
Insert	$\mathcal{O}(n)$
Delete	$\mathcal{O}(n)$
Increase size	—

Listeler

Günlük hayatta listeler

- Alışveriş listesi
- Yapılacak işler listesi
- Öğrenci listesi
- Telefon listesi

Bu listelere eleman ekleme işlemi

- Diğerlerinden daha zor olan var mı?
- Neden?

Bu listedeki elemanları eleme (silme) işlemi

- Farklı durumu olan var mı?

Listeler

Programlama açısından liste

- Birbirleriyle ilişkili veri toplulukları
- Değişik veri yapı türleriyle gösterilebilir
- Üzerlerinde veri yapısına özgü yöntemlerle işlemler yapılabilir
 - Ekleme
 - Silme
 - Arama

Programlamada Listeler

Doğrusal Listeler (Diziler)

- Aynı türde elemanlar
- Bellekte artan sırada
- Program başında tanımlanır ve bellekteki boyutu belirlenir
 - Çalışma anında boyutu değiştirilemez
 - Çok büyük tanımlanırsa gereksiz bellek kullanımı olabilir
- Sona eleman eklemek gerekirse
 - ?
- Araya eleman eklemek gerekirse
 - Ekleme konumundan sonraki tüm elemanlar bir öteye kayar
- Eleman silmek gerekirse
 - Silme konumundan sonraki tüm elemanlar bir öne kayar
- Sıralama işleminde tüm elemanların yer değiştirmesi gerekebilir

Bağlantılı Listeler

Bağlı Liste

- Bellekte elemanları ardışık olmayan listeler

Her eleman bir sonraki elemanın bellekte nerede olduğu bilgisini tutar

- Bu şekilde elemanlar dolaşılabilir

Bağlı Liste elemanları 2 kısımdan oluşur

- Verinin tutulduğu kısım
- Bir sonraki elemanın adresinin tutulduğu kısım

O zaman bir bağlantılı liste elemanı nasıl oluşturulabilir?

Bağlantılı Listeler

İlk elemanı nasıl gösterebiliriz?

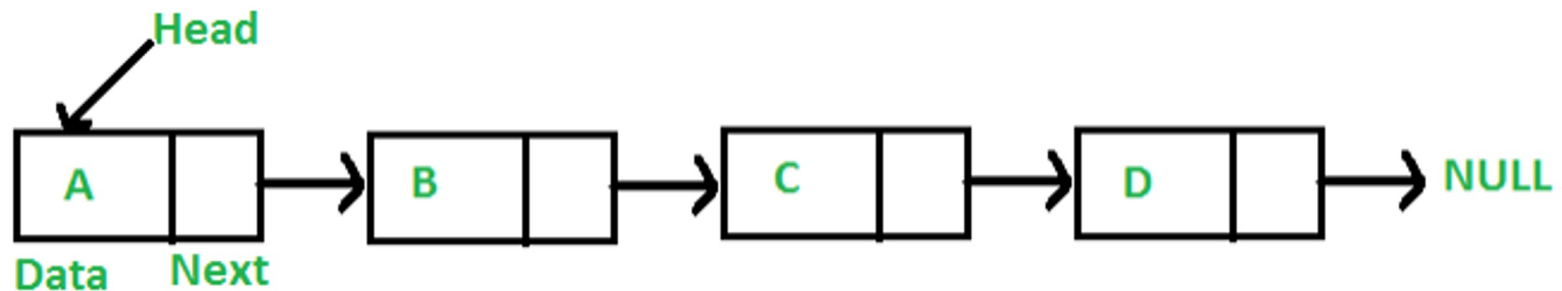
- İlk elemanın yeri o yapı türünden bir göstericide tutulabilir
- O göstericiden başlayarak tüm elemanlara erişilebilir

Son eleman kimi gösterecek?

- NULL?

Bağlantılı liste elemanlarının her biri bir nesne

- Bazı elemanları o elemanın tutacağı veri/bilgilerin değerlerini
- Bir elemanı ise bir sonraki elemanın adresini tutar



Bağlantılı Listeler

Tek Yönlü Bağlantılı Liste (One-Linked Liste):

- Düğümler arasında tek yönlü bağlantı vardır;
- listenin başından sonuna doğru hareket edilebilir.
- Ekleme, arama, listeleme gibi işlemlerin karmaşıklığı olur.

Çift Yönlü Bağlantılı Liste (Double-Linked List):

- Düğümler arasındaki bağlantı iki yönlüdür;
- Bir düğüm hem bir sonraki hem de bir önceki düğümü işaret eder.
- Dolayısıyla iki yönlü hareket edilebilir.
- İki işaretçi değişken kullanılırsa, arama ve listeleme maliyetleri yine $O(n)$ olurken ekleme maliyeti $O(1)$ olur.

Bağlantılı Listeler

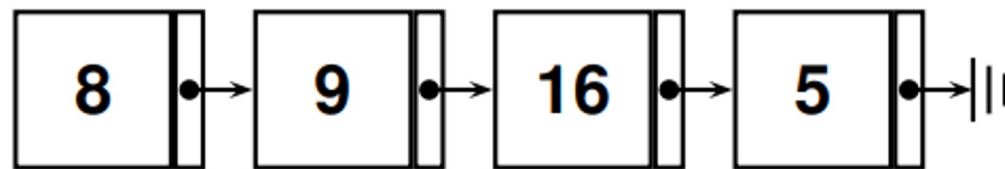
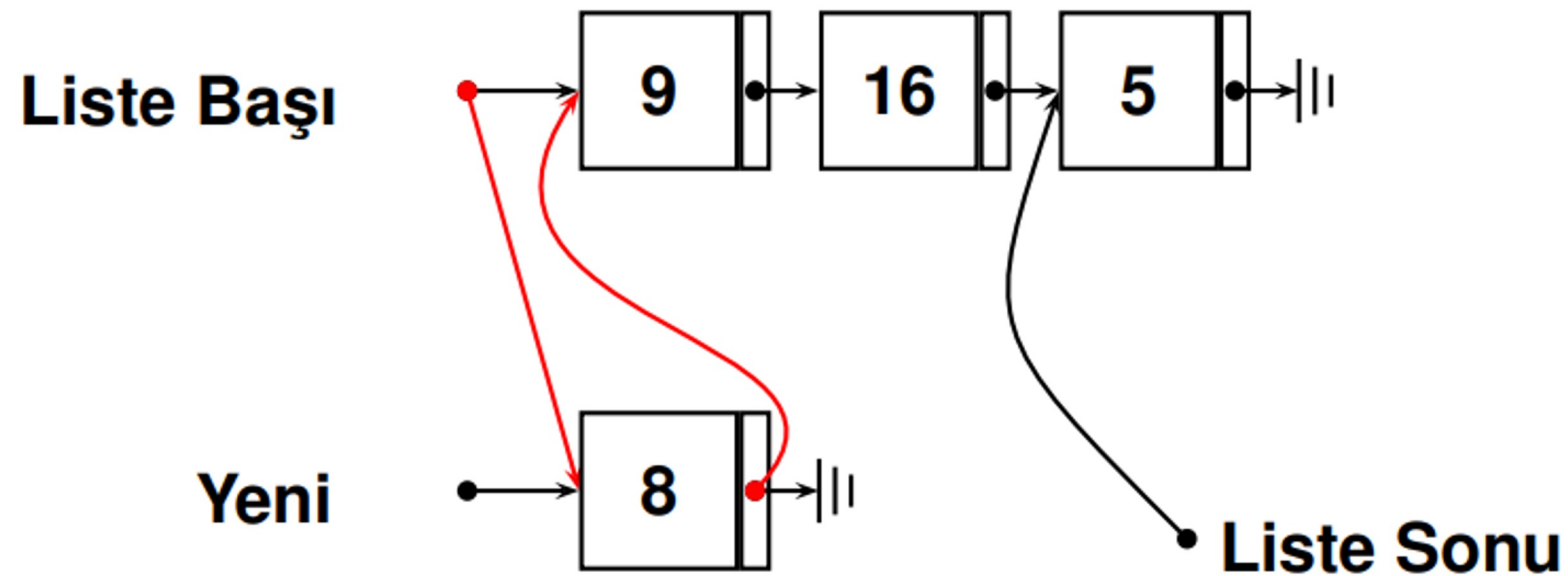
Çevrimsel Bağlantılı Liste (Circular Linked List):

- Çevrimsel bağlantılı listelerde gibi düğümler arasında çevrimsel bir bağlantı vardır;
- Eğer elemanlı bir bağlantılı listede adım hareket edilirse, yine, listenin başladığı noktaya dönlür.

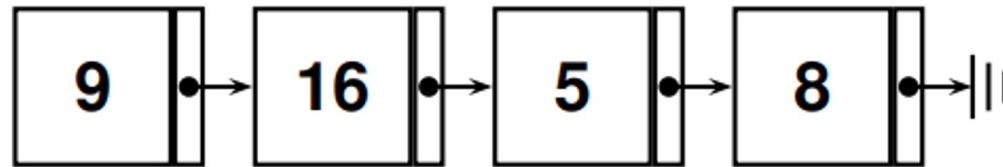
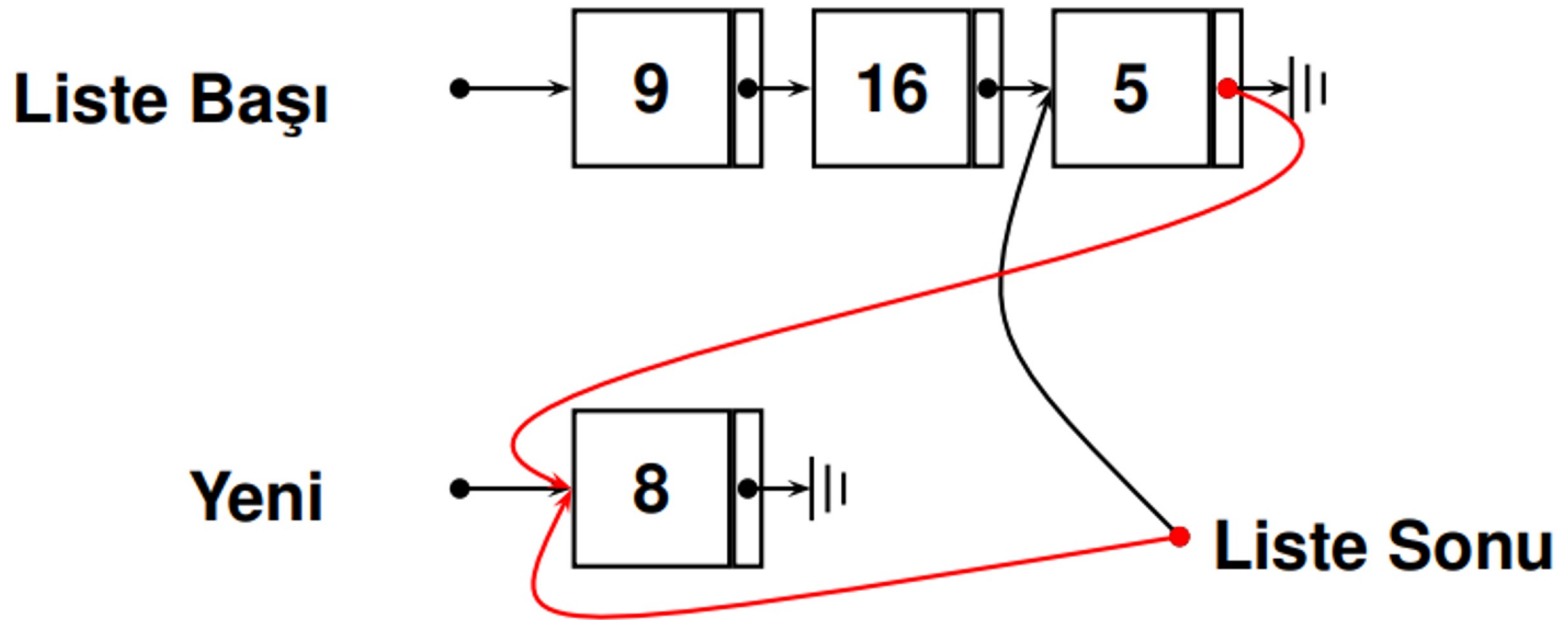
Karma Bağlantılı Liste (Hybrid Linked List):

- Çeşitli bağlantılı listelerin bir araya getirilmesiyle oluşturulan listeler
- Gereksinime göre, diğer bağlantılı listelerin özellikleri alınır ve yeni bir bağlantılı liste modeli oluşturulur

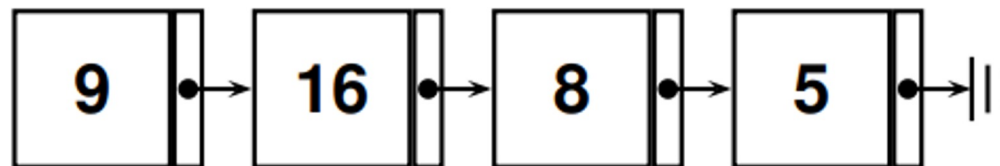
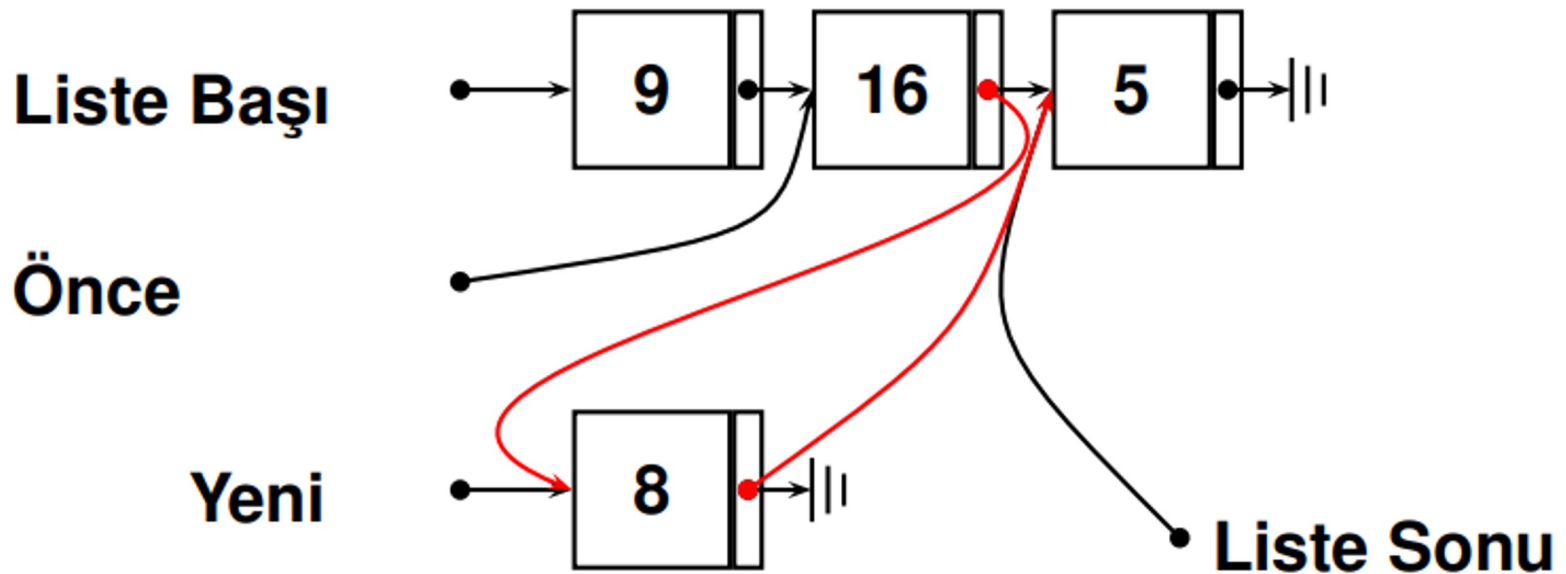
Bir Bağlantılı Listenin Başına Eleman Ekleme



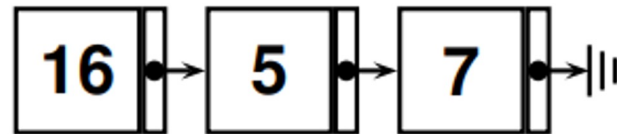
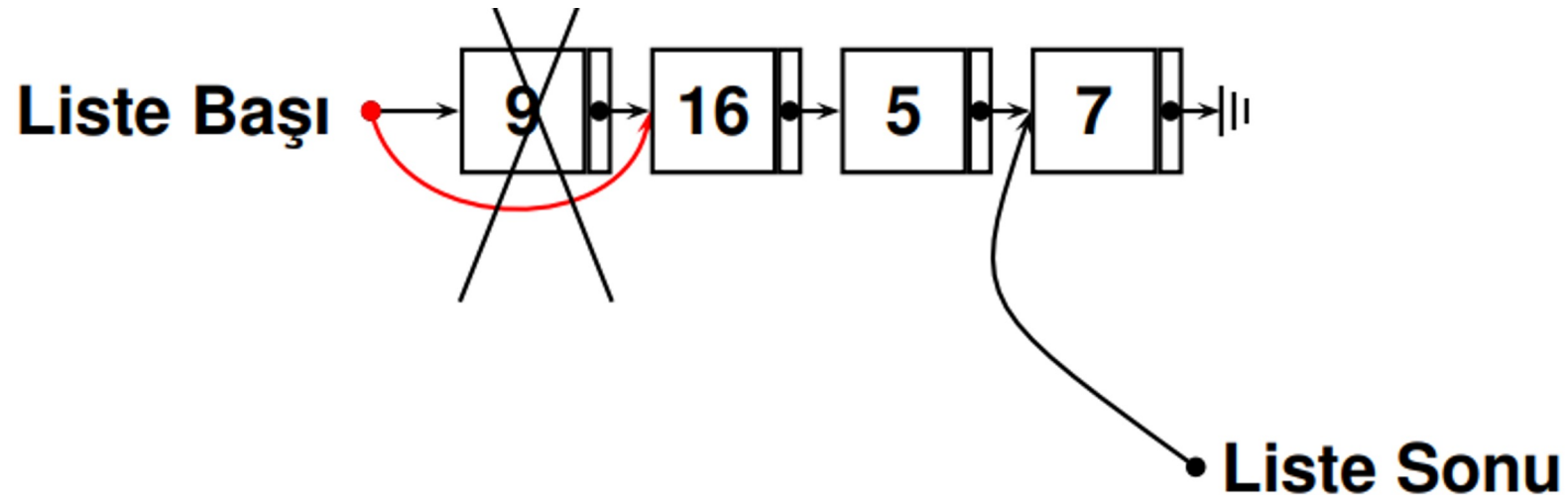
Bir Bağlantılı Listenin Sonuna Eleman Ekleme



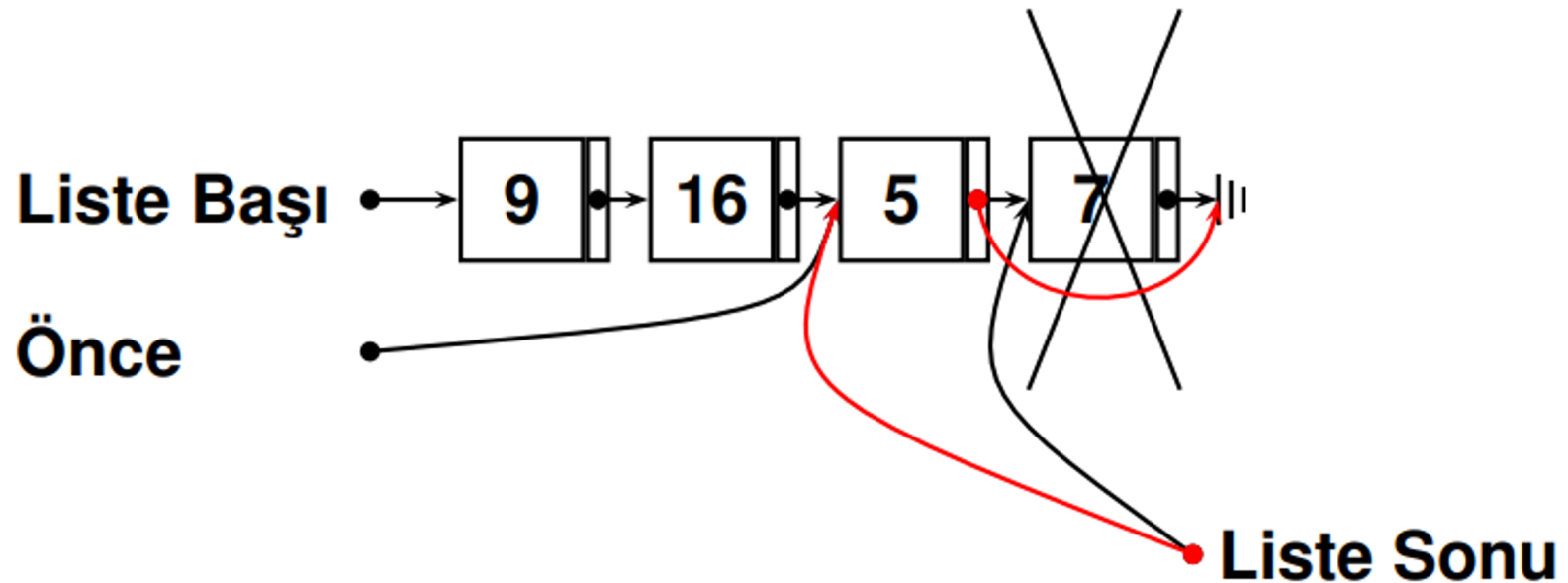
Bir Bağlantılı Listenin Arasına Eleman Ekleme



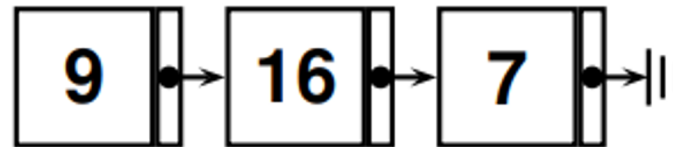
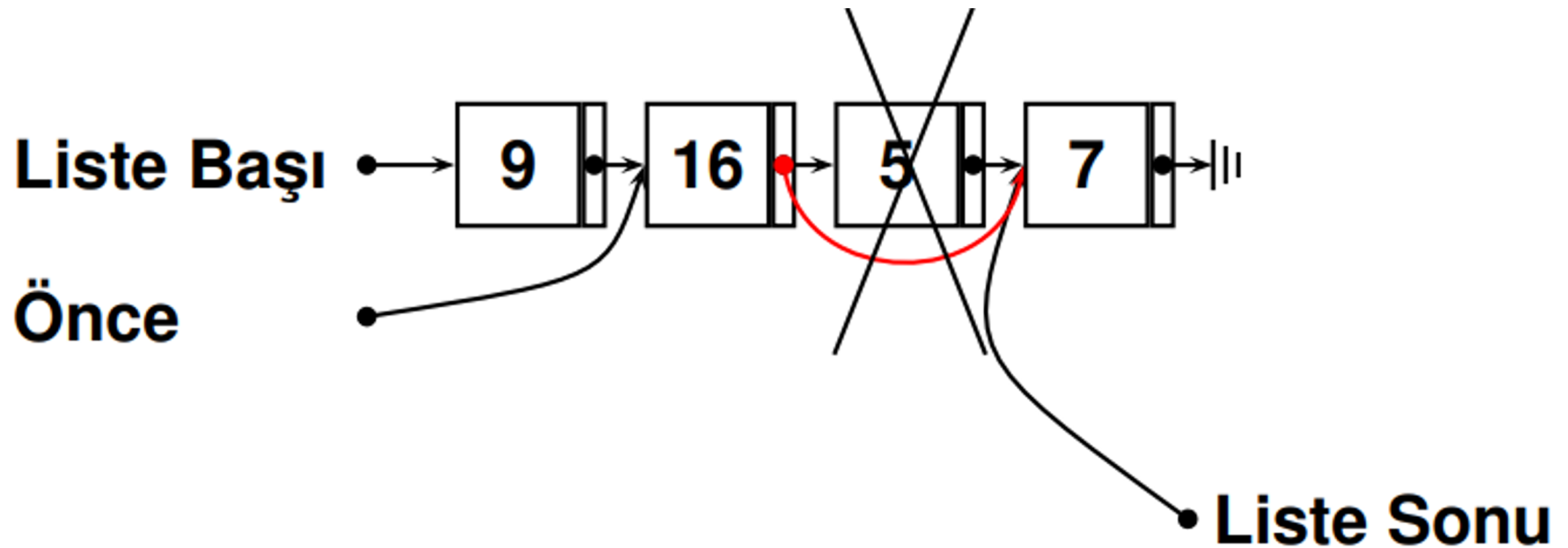
Bir Bağlantılı Listenin İlk Elemanını Silme



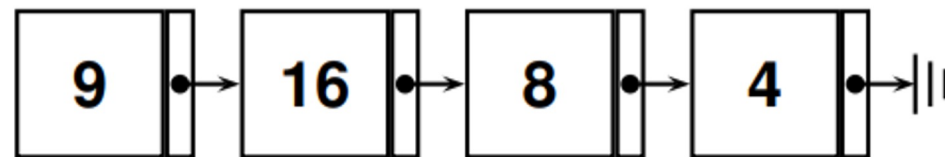
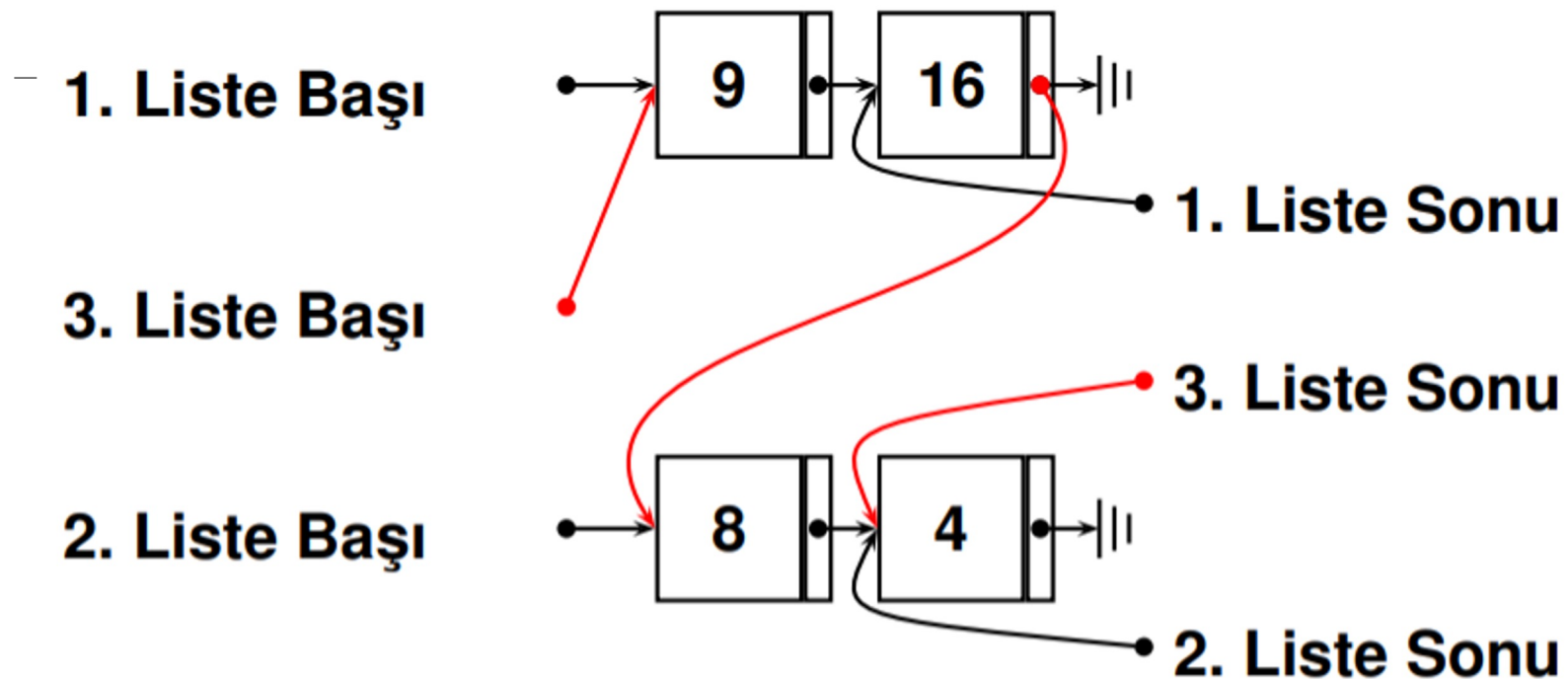
Bir Bağlantılı Listenin Son Elemanını Silme



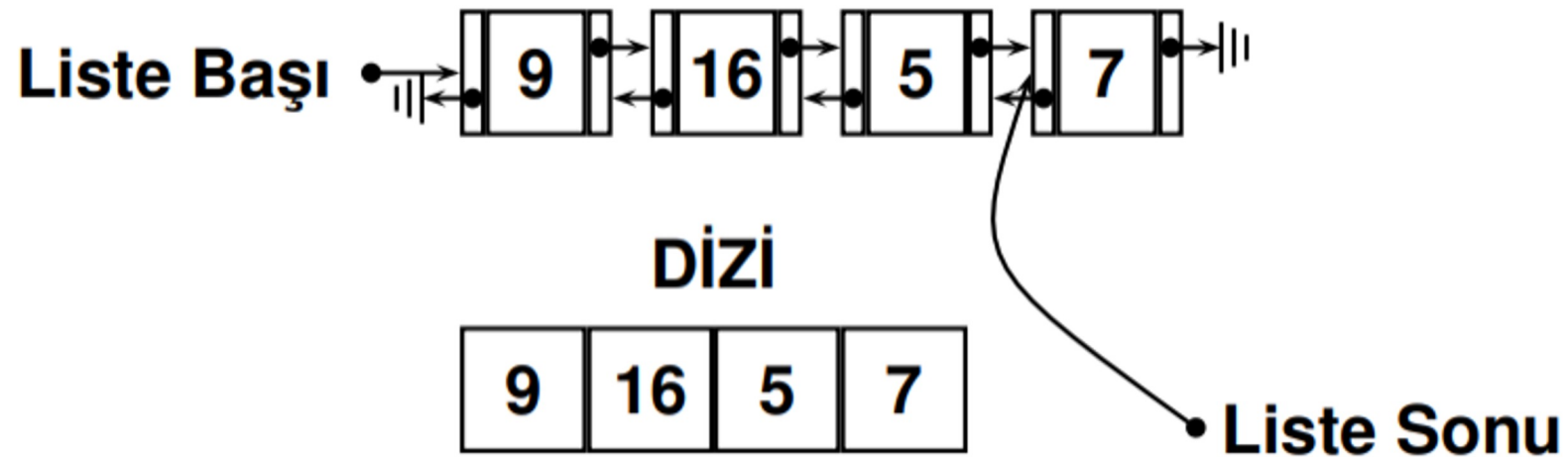
Bir Bağlantılı Listenin Ara Elemanını Silme



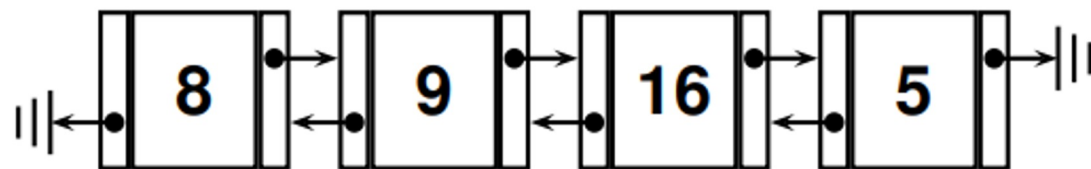
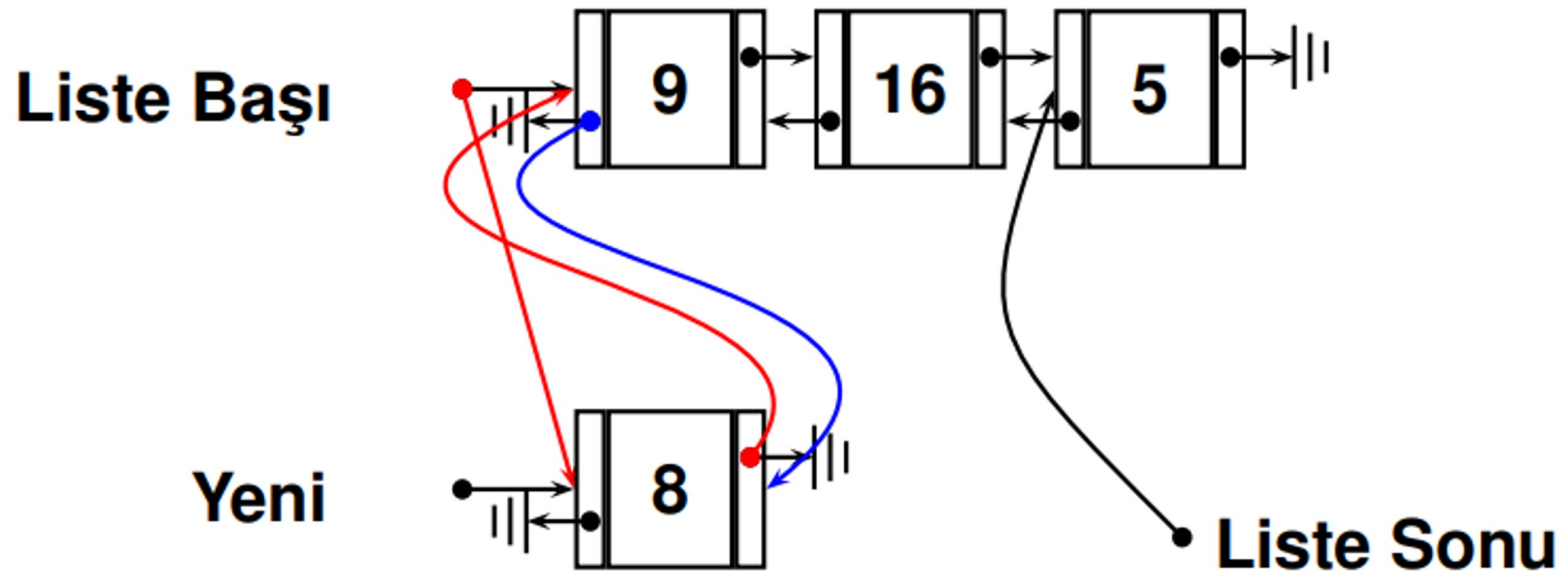
İki Bağlantılı Listeyi Birleştirme



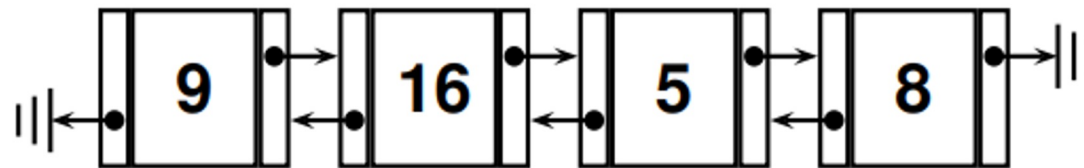
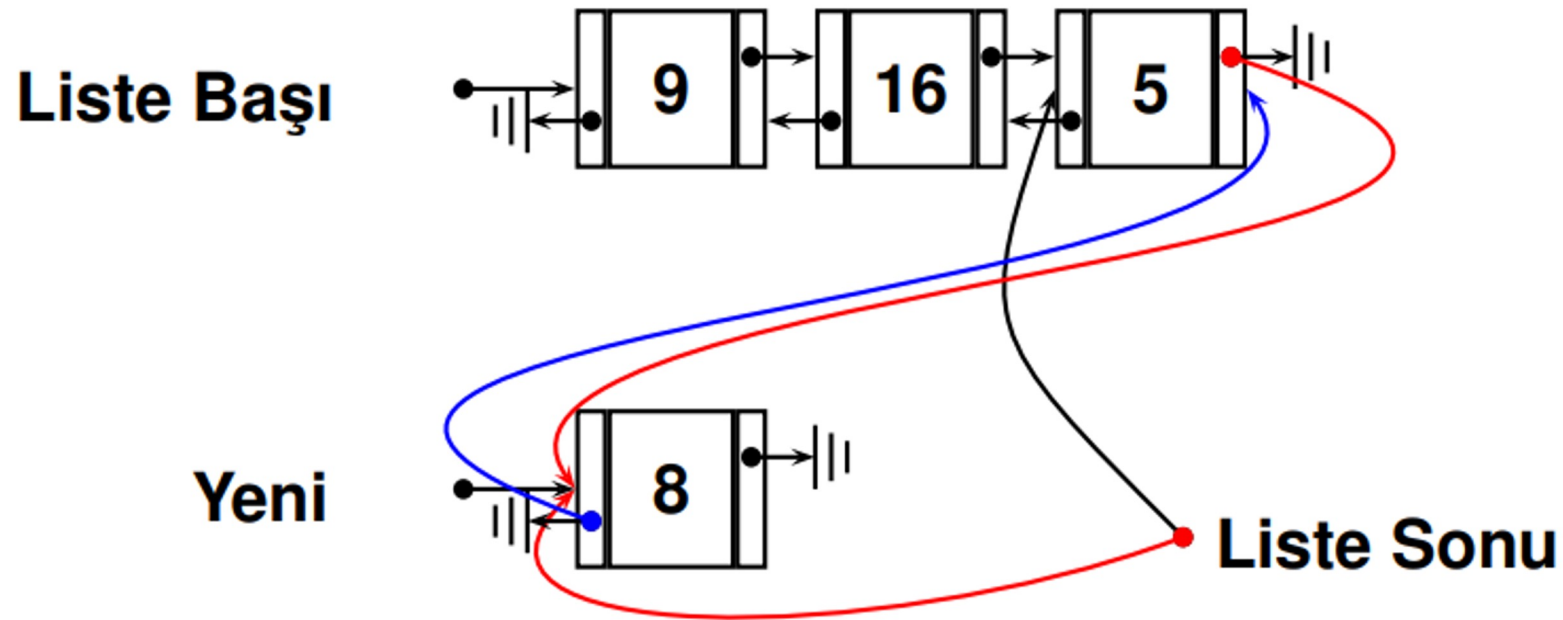
Çift Bağlantılı Liste



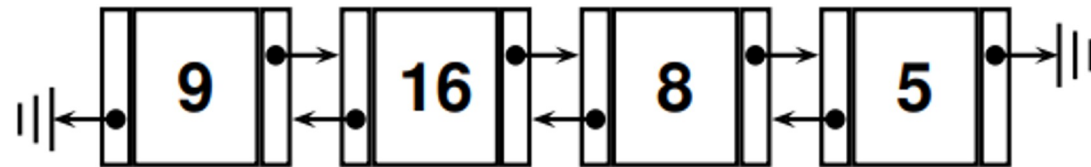
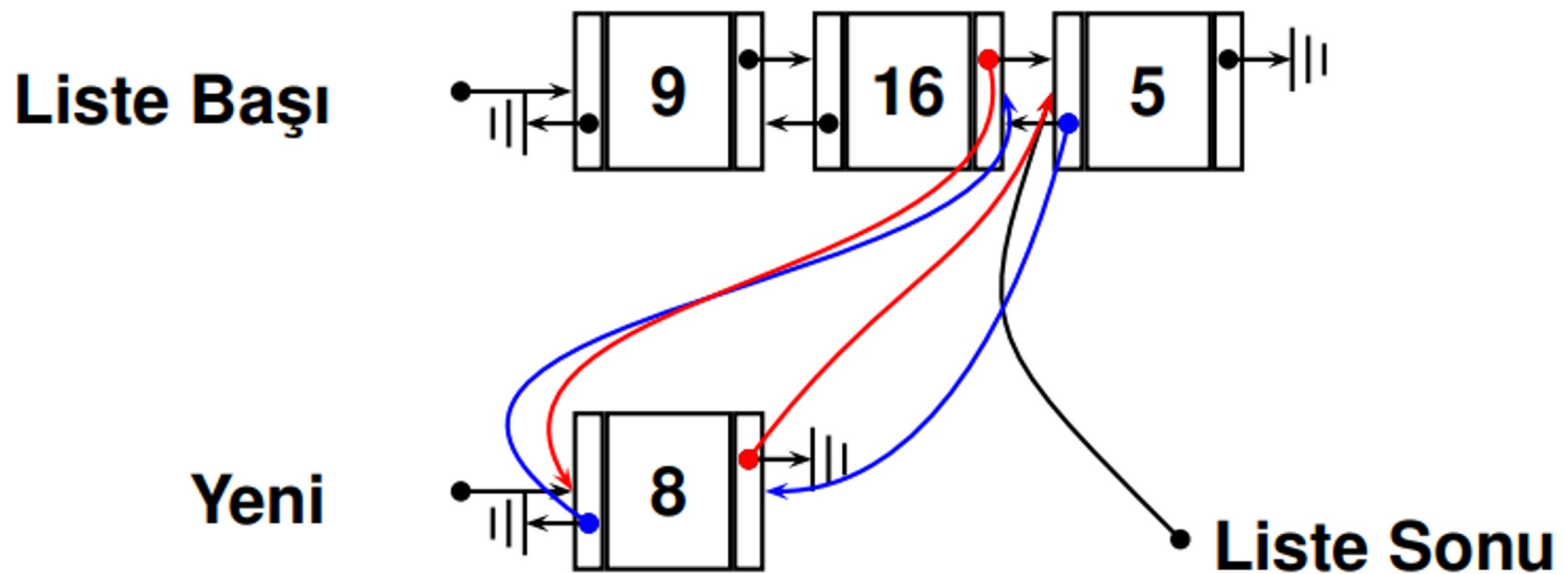
Çift Bağlantılı Listenin Başına Eleman Ekleme



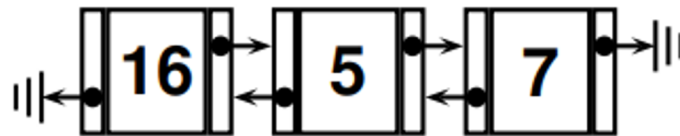
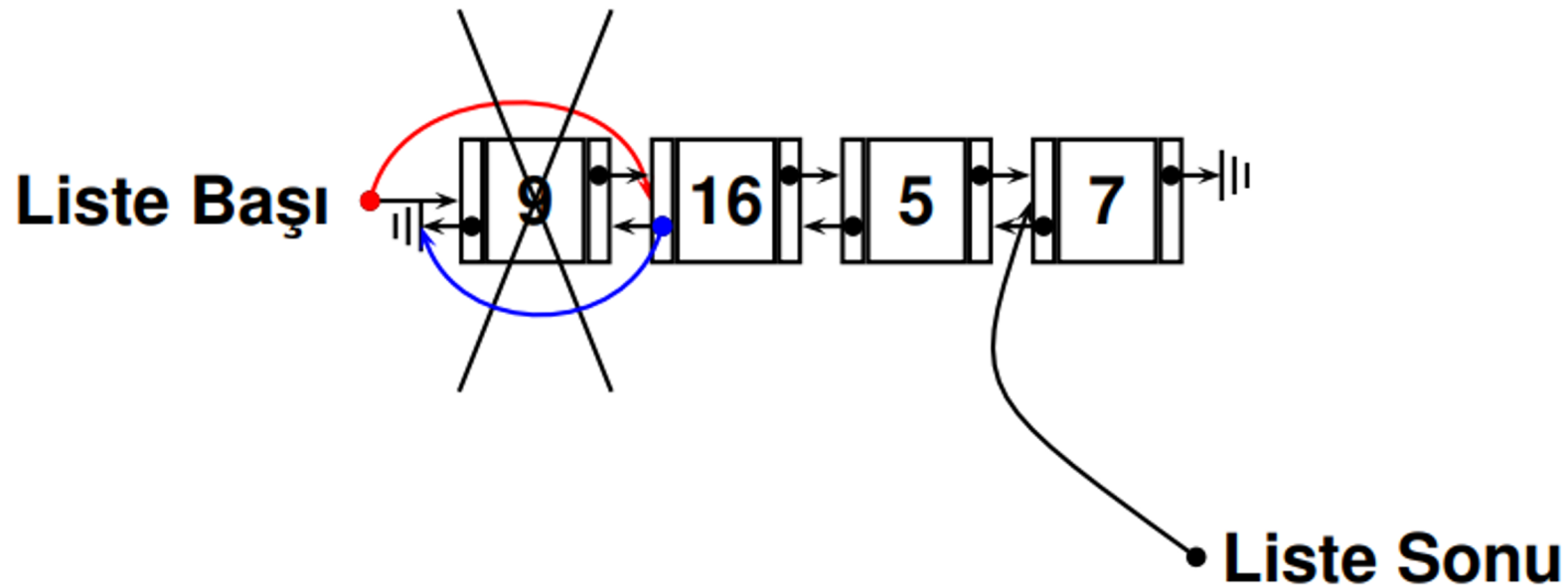
Çift Bağlantılı Listenin Sonuna Eleman Ekleme



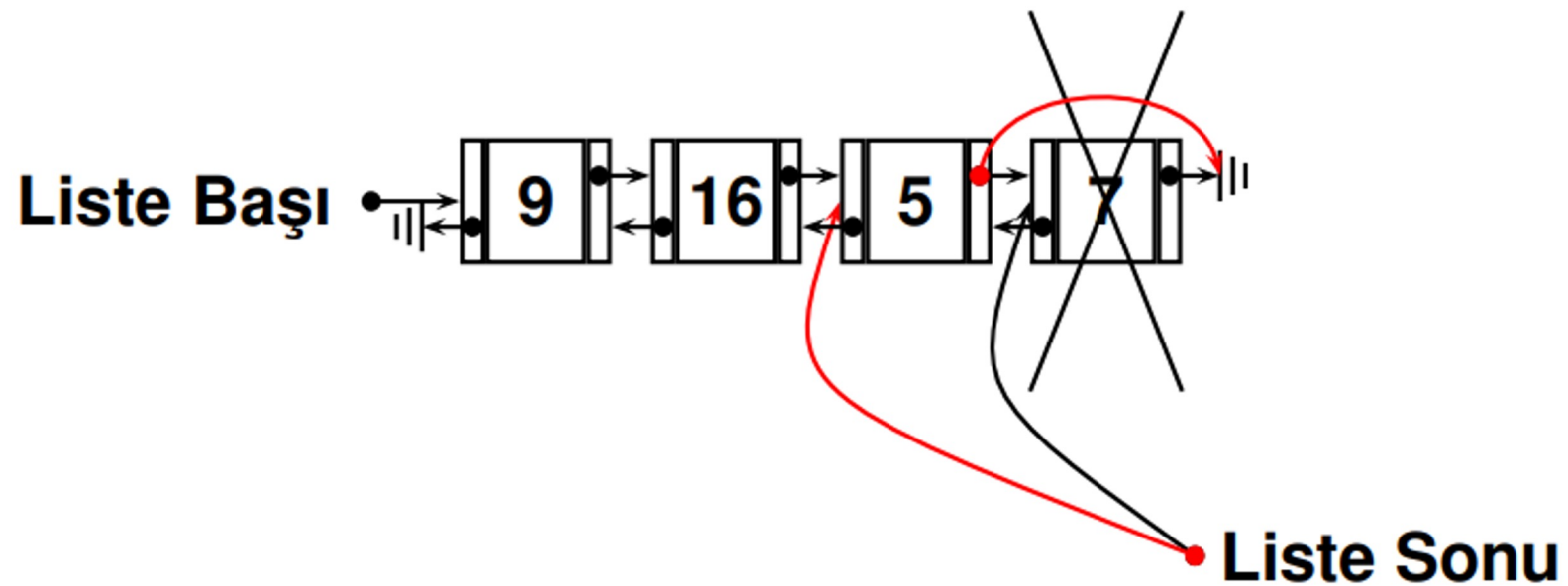
Çift Bağlantılı Listenin Arasına Eleman Ekleme



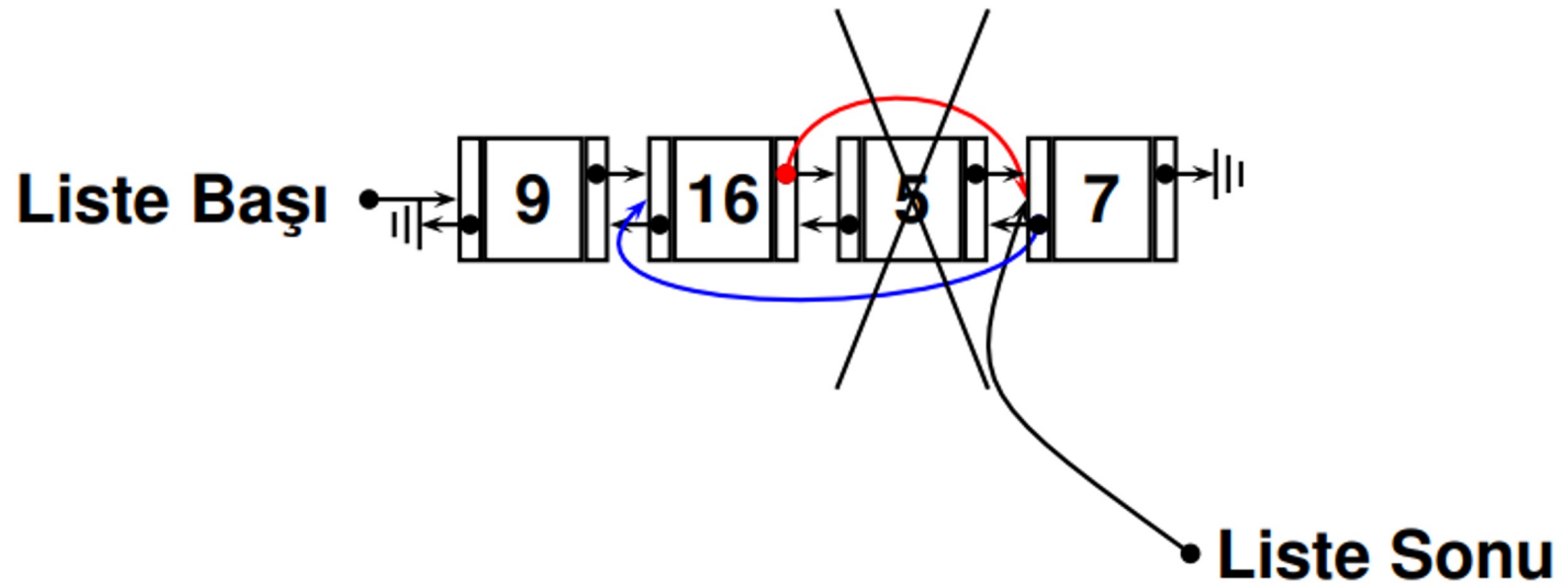
Çift Bağlantılı Listenin İlk Elemanını Silme



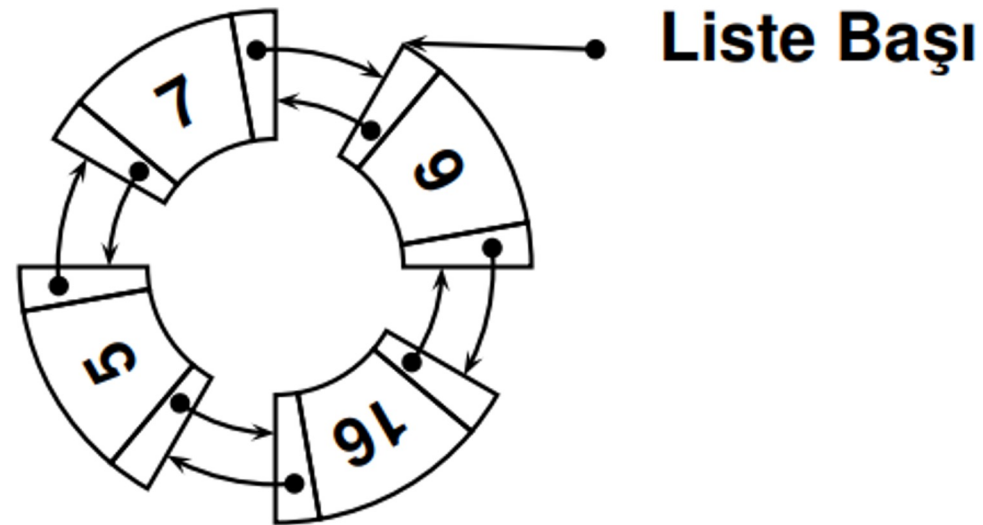
Çift Bağlantılı Listenin Son Elemanını Silme



Çift Bağlantılı Listenin Ara Elemanını Silme



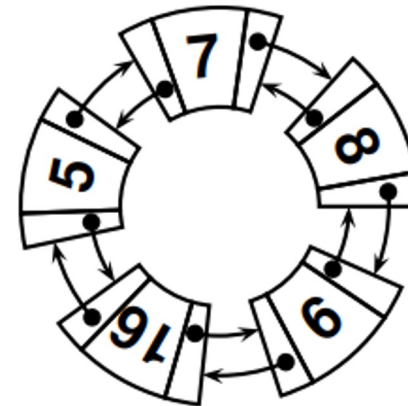
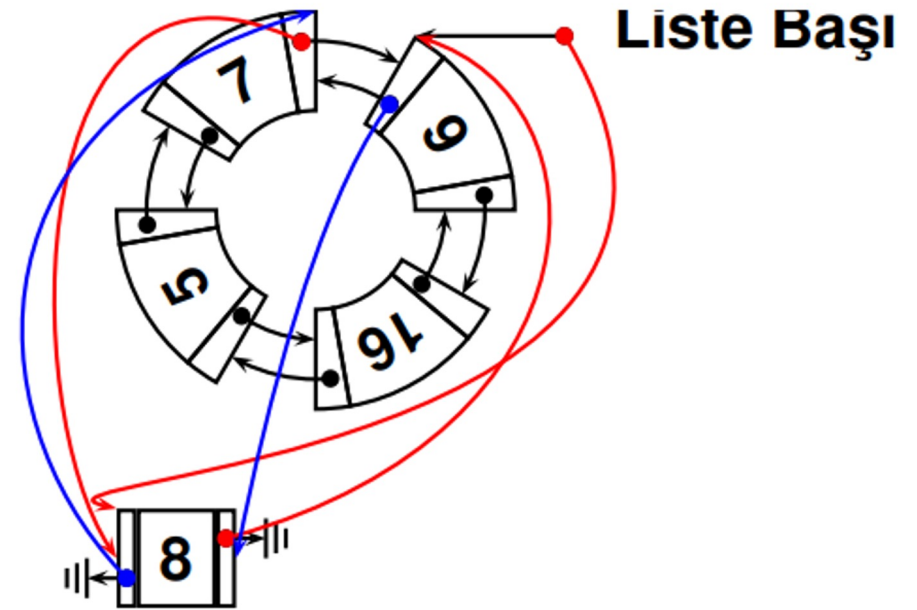
Çevrimysel Bağlantılı Liste



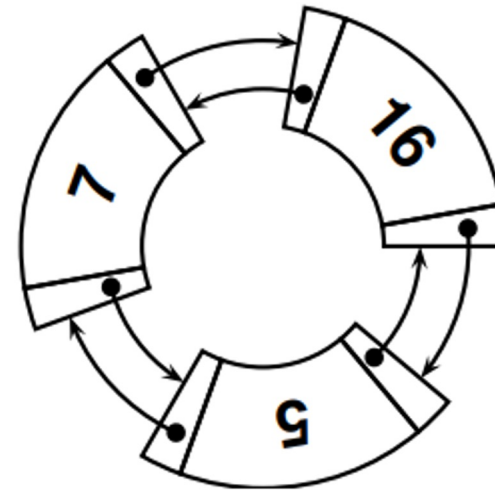
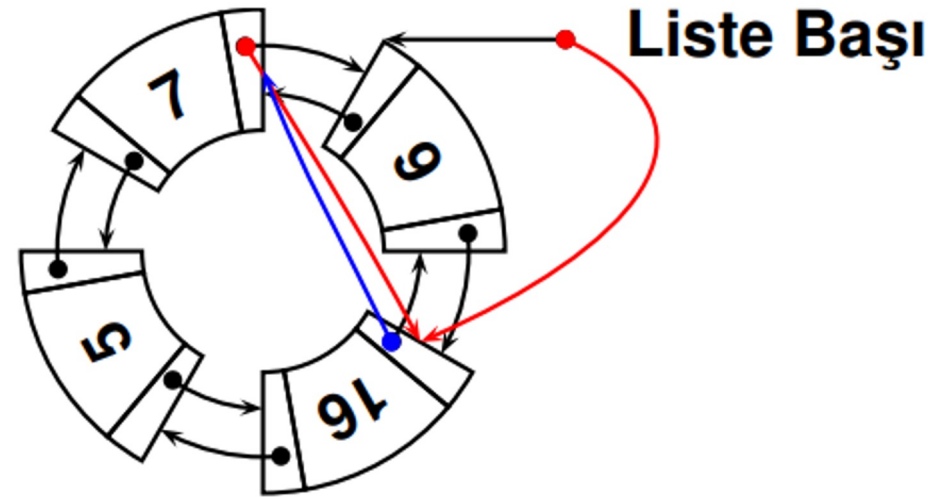
Dizi

9	16	5	7
---	----	---	---

Çevrimsel Bağlantılı Listenin Başına Ekleme

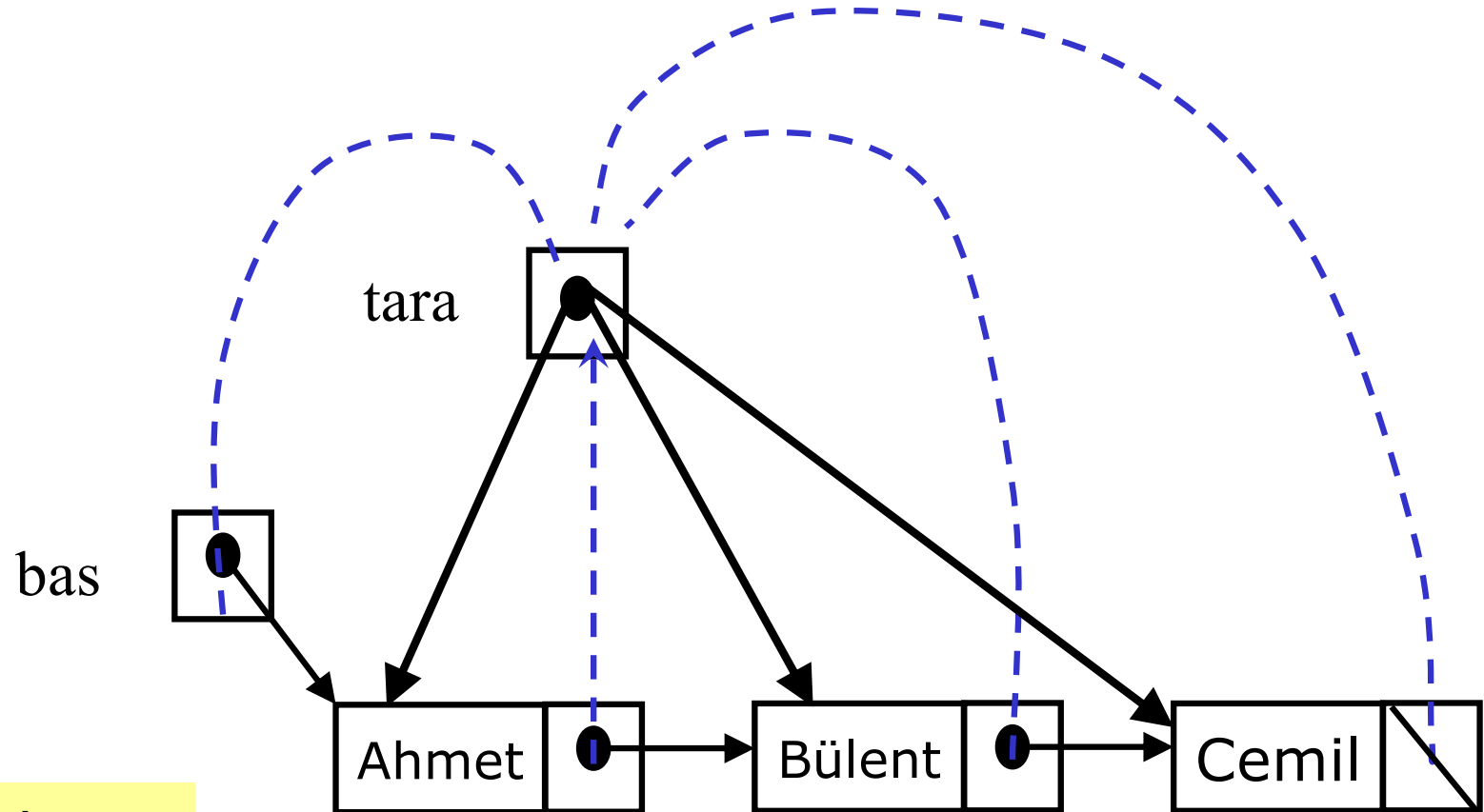


Çevrimsel Bağlantılı Listenin İlk Elemanını Silme



listeyaz() Fonksiyonu

listeyaz



Ekran çıktısı :

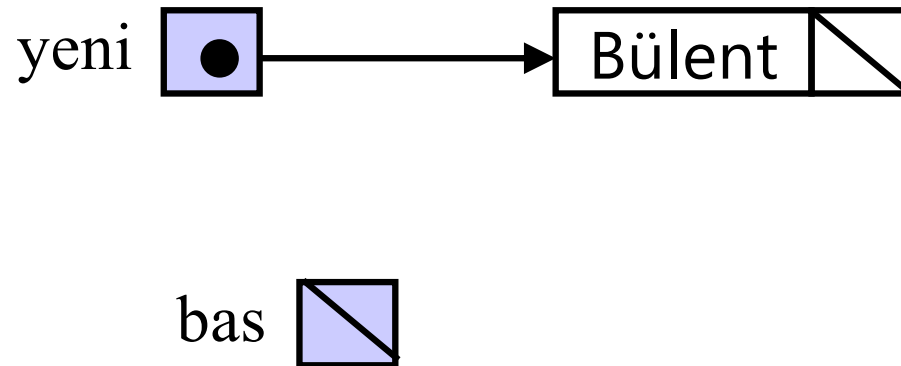
Ahmet
Bülent
Cemil

ekle() Fonksiyonu

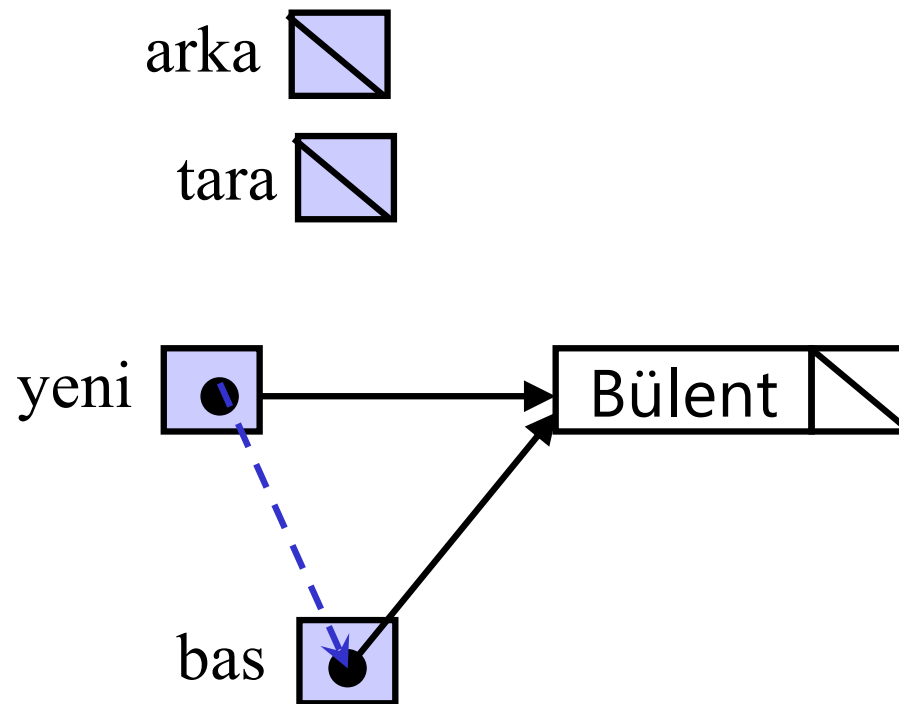
- Durum1 : Boş listeye ekle
- Durum2 : Listenin başına ekle
- Durum3 : Araya ekle
- Durum4 : Listenin sonuna ekle

Durum1 : Boş listeye ekle

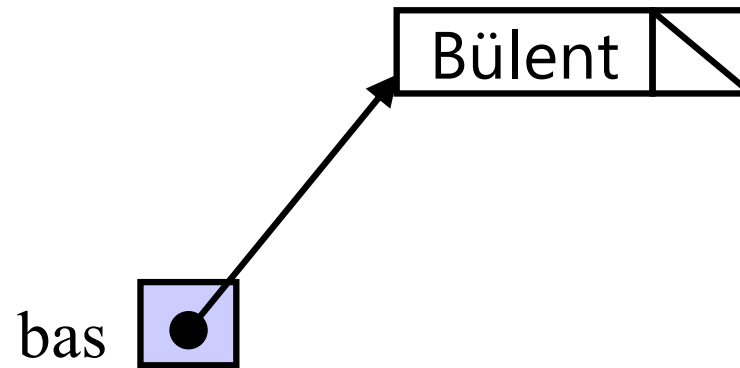
Boş listeye ekle (1)



Boş listeye ekle (2)

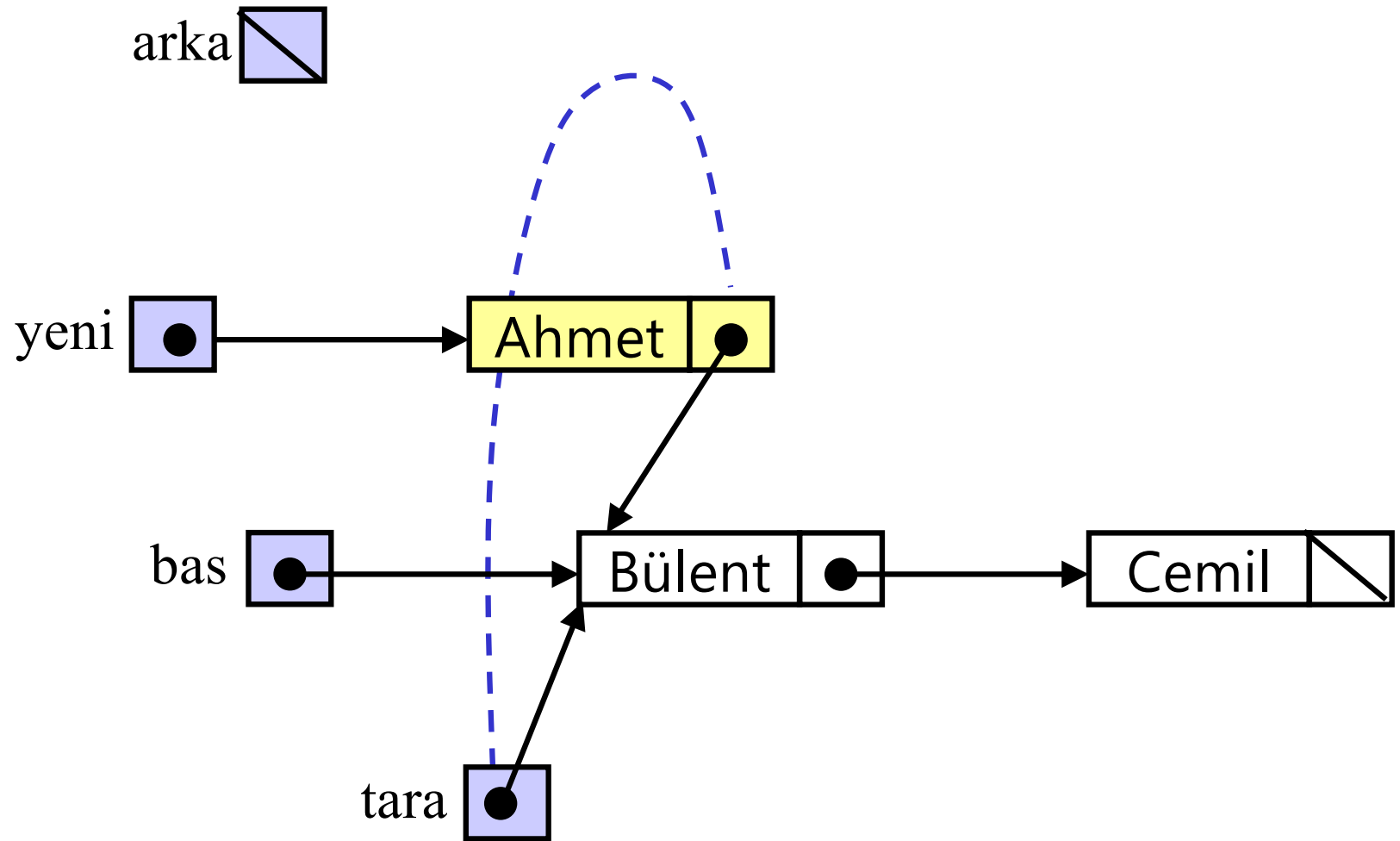


Boş listeye ekle (3)

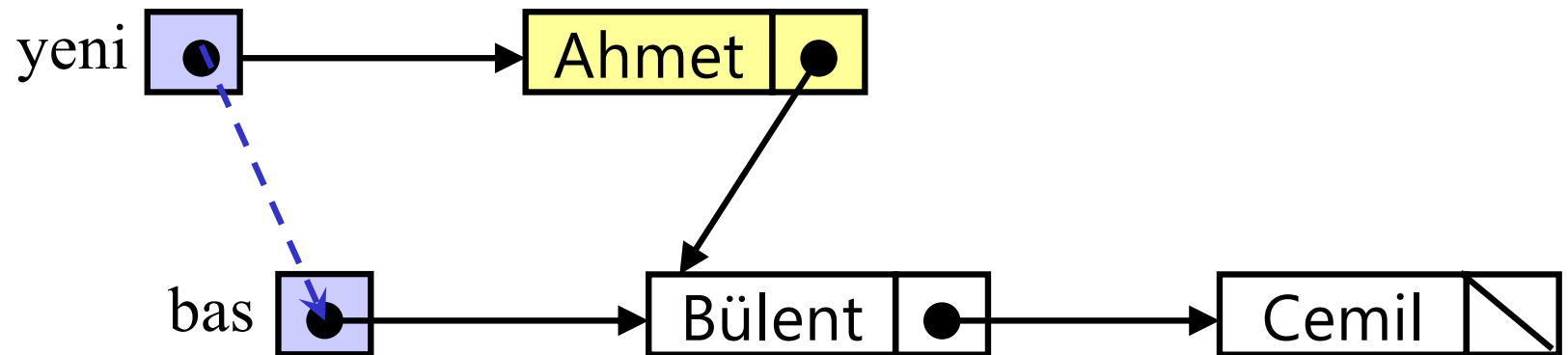


Durum2 : Listenin başına ekle

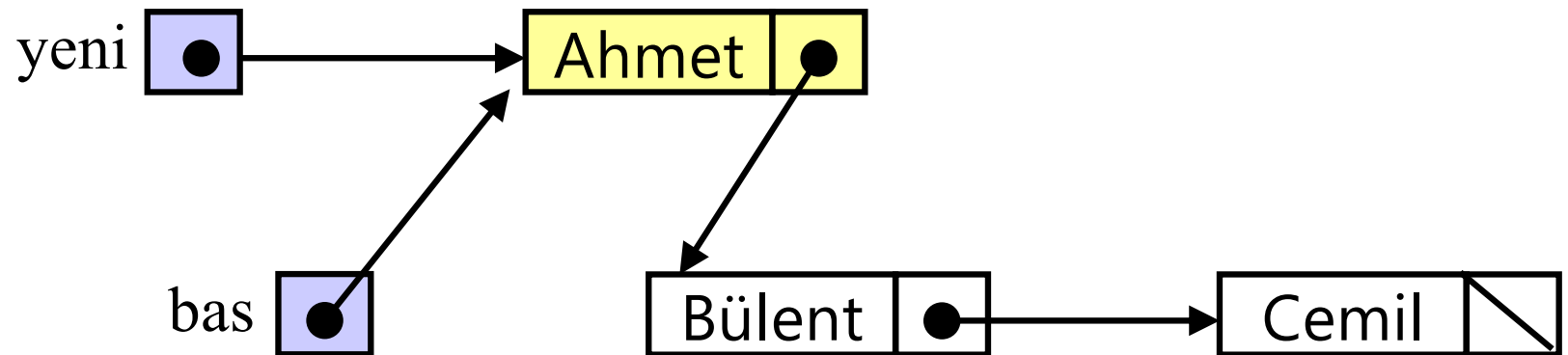
Listenin başına ekle (1)



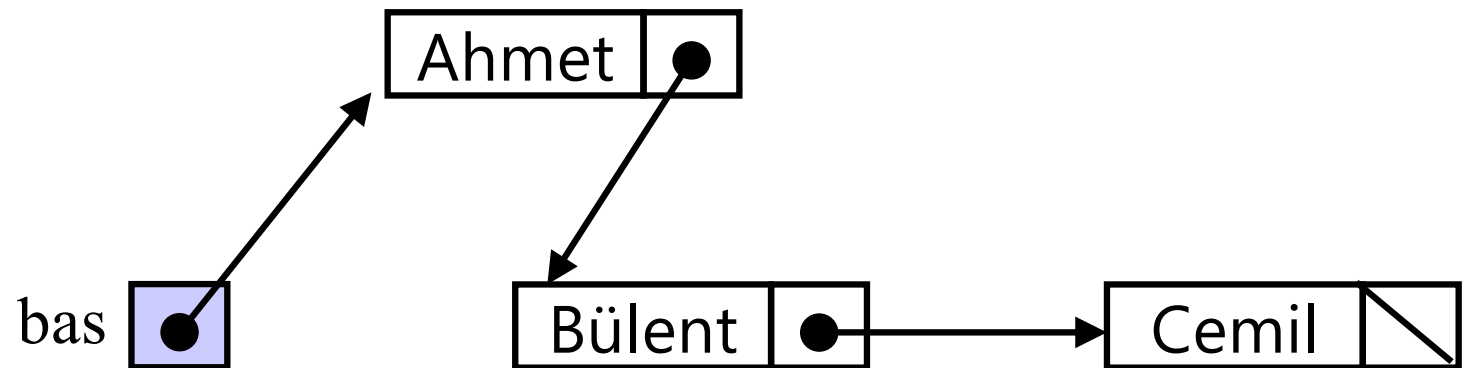
Listenin başına ekle (2)



Listenin başına ekle (3)

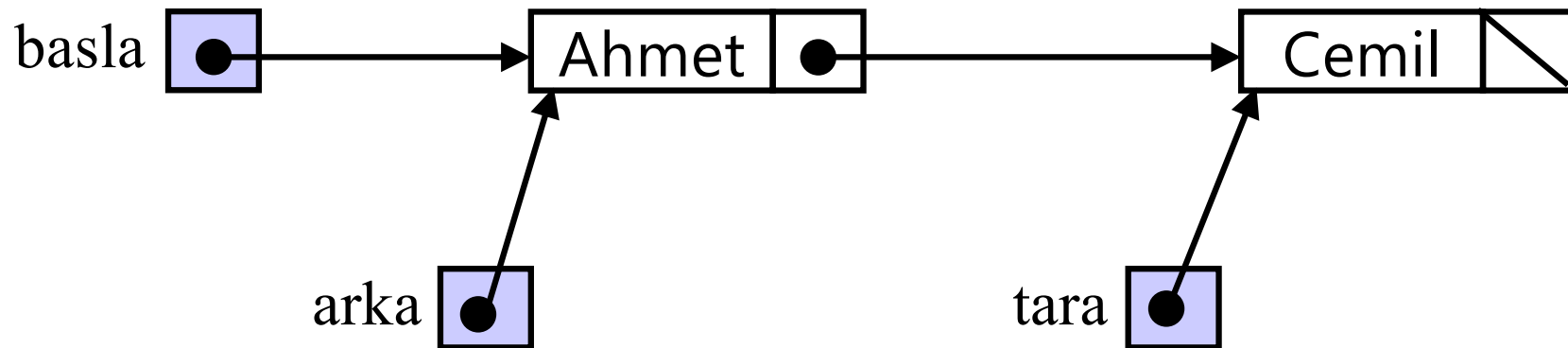
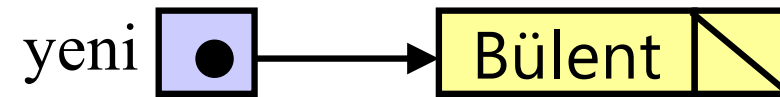


Listenin başına ekle (4)

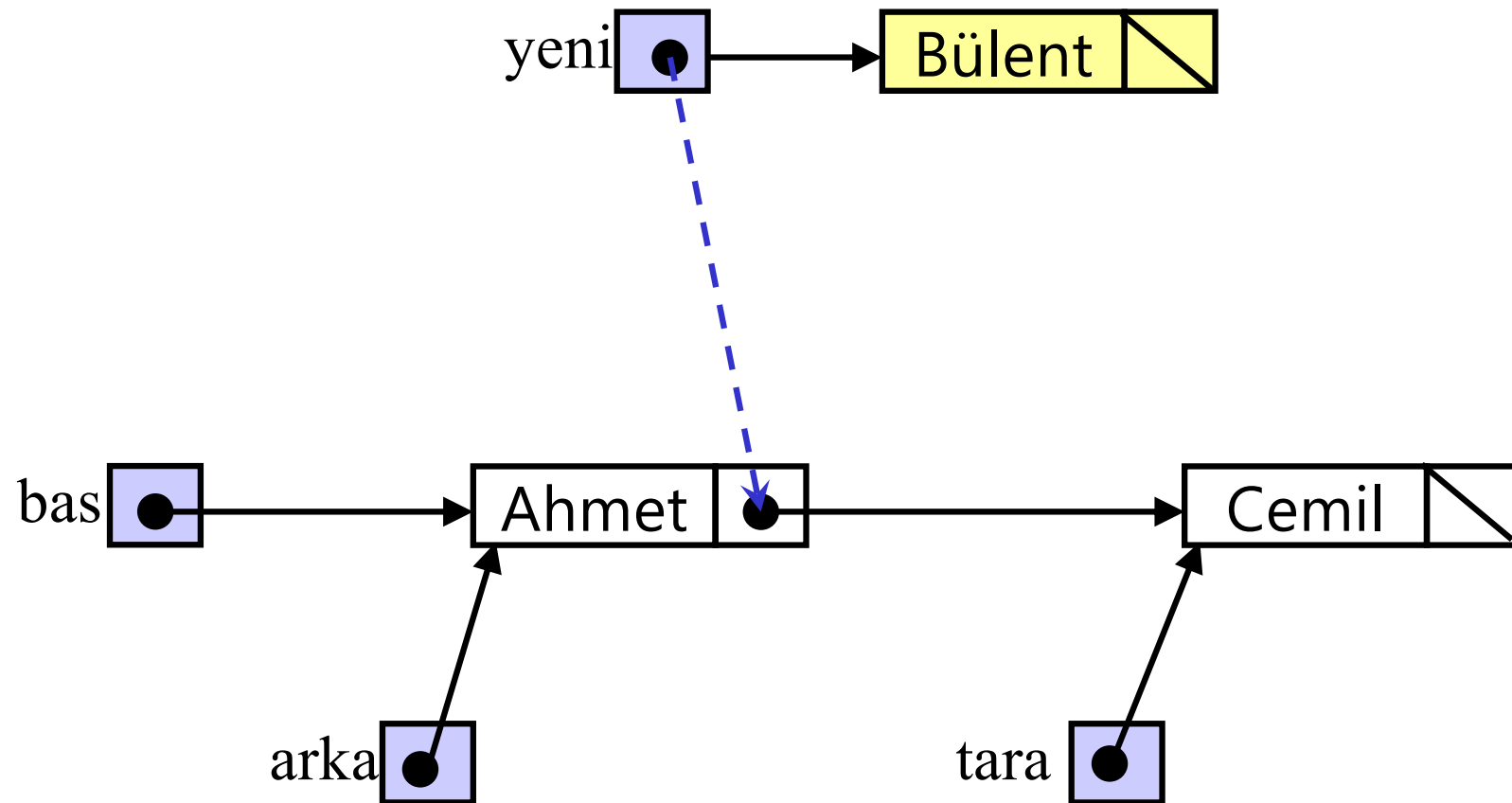


Durum3: *Araya ekle*

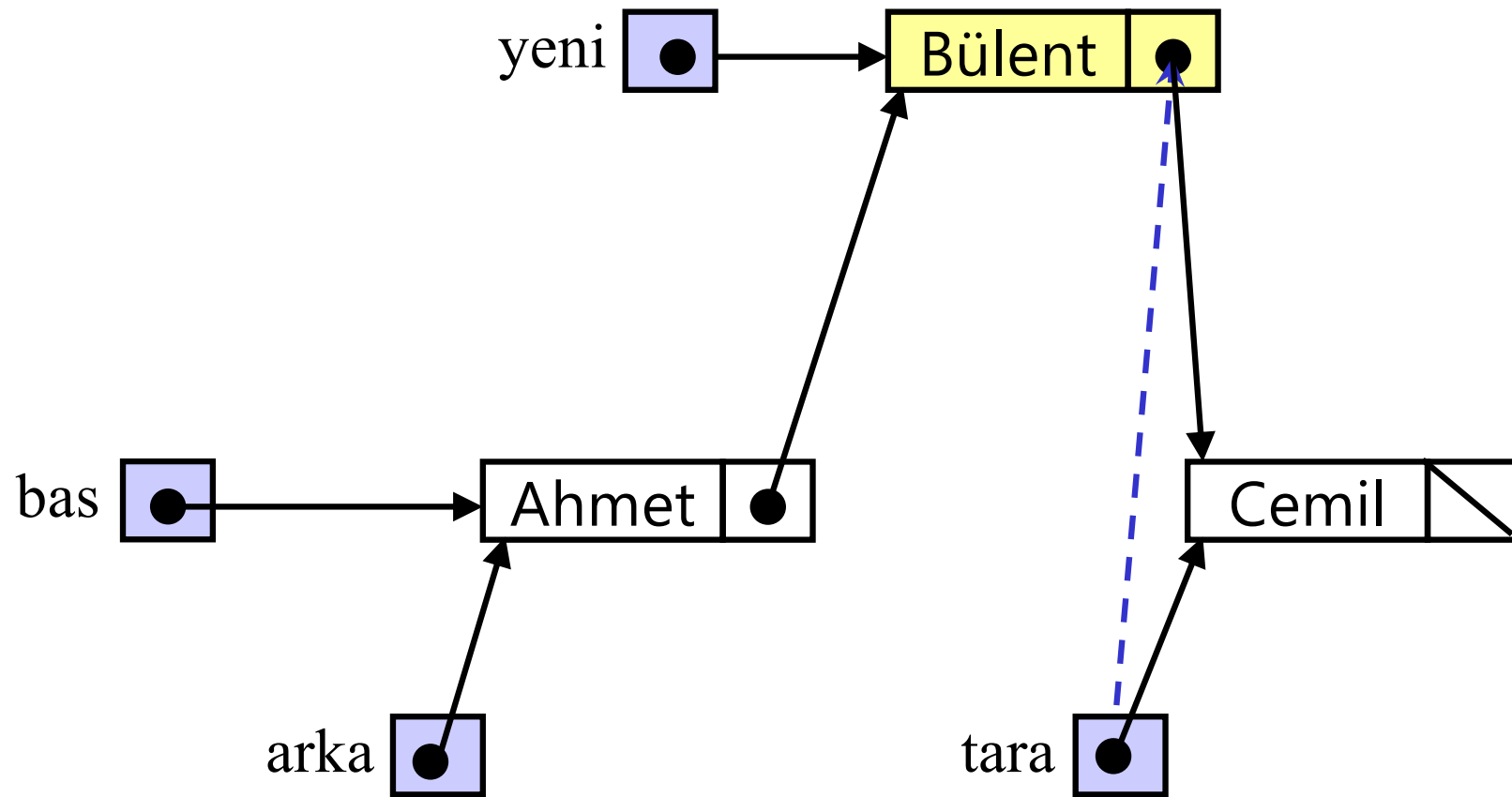
Araya ekle (1)



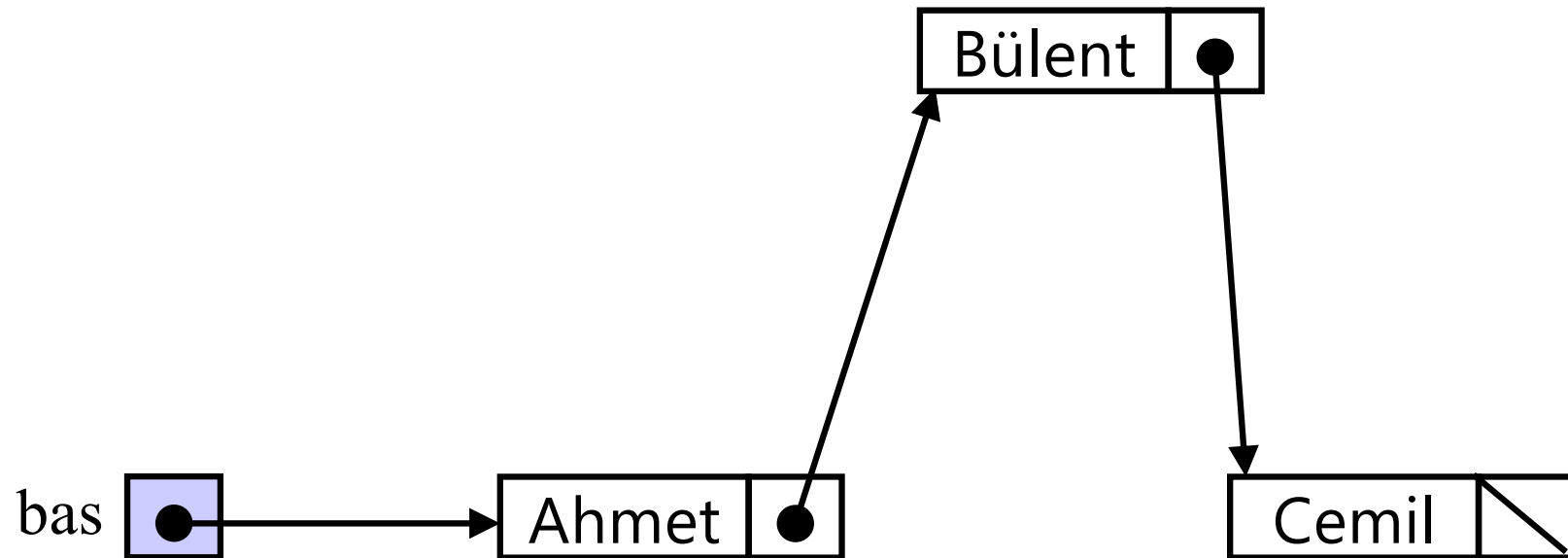
Araya ekle (2)



Araya ekle (3)

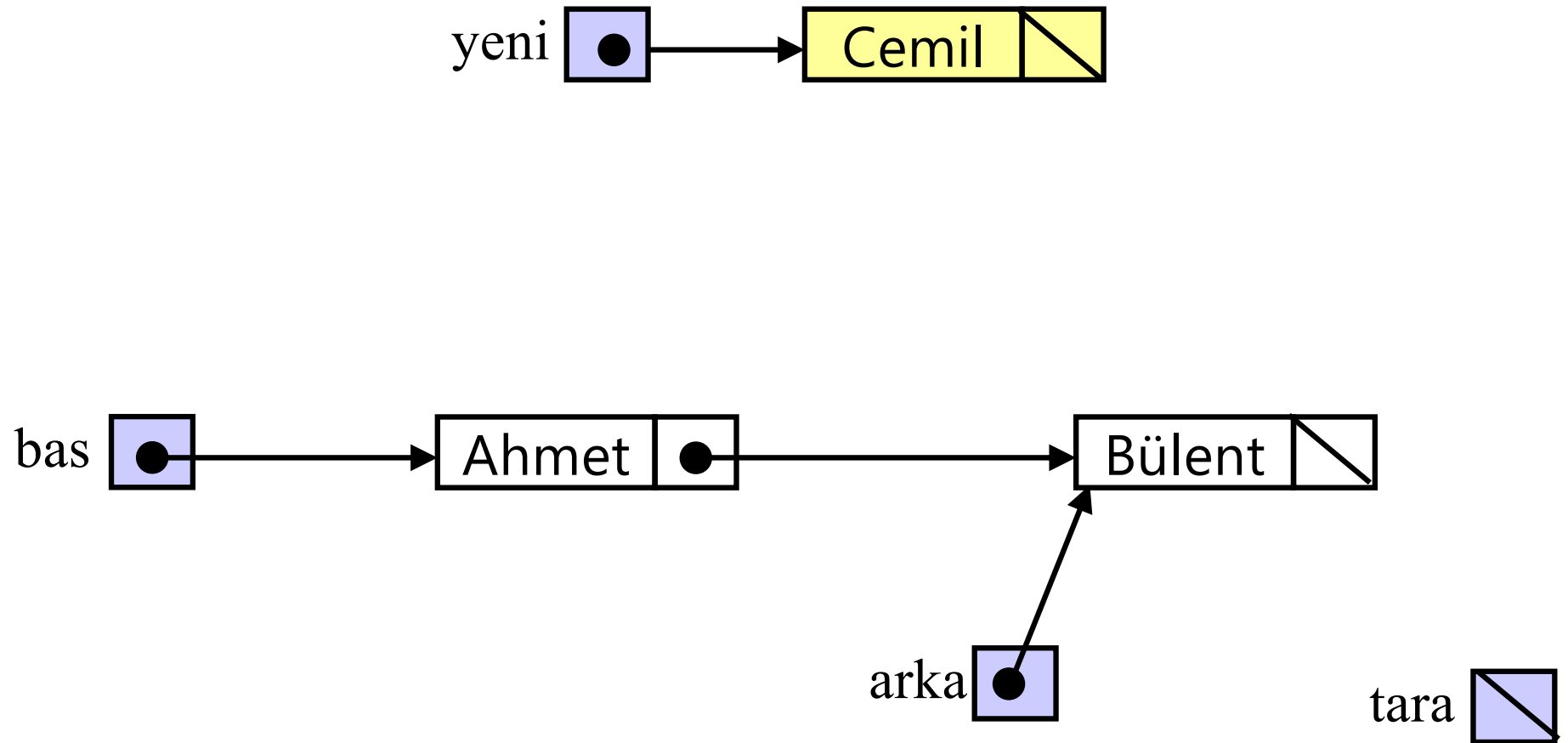


Araya ekle (4)

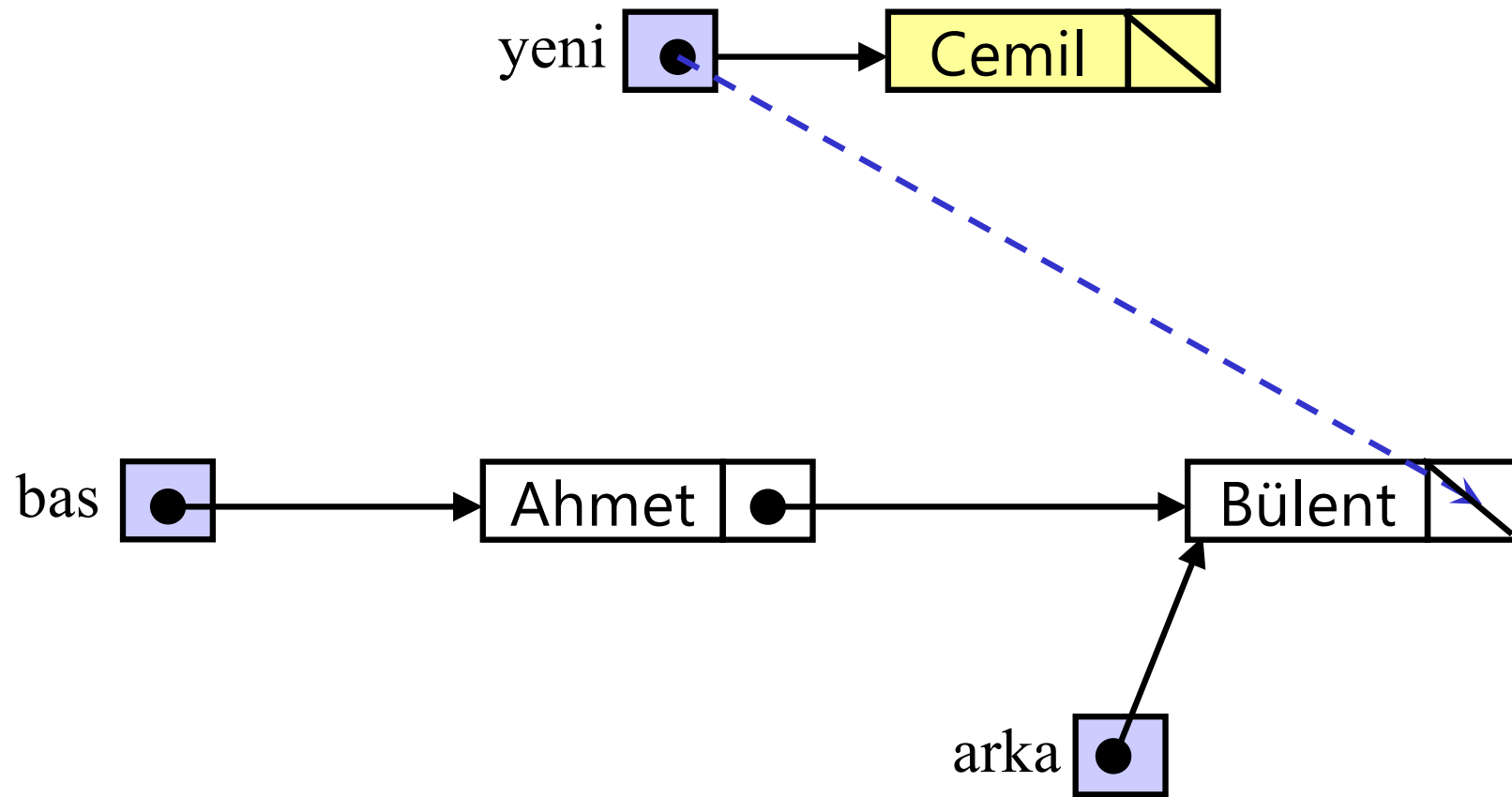


Durum4 : Listenin sonuna ekle

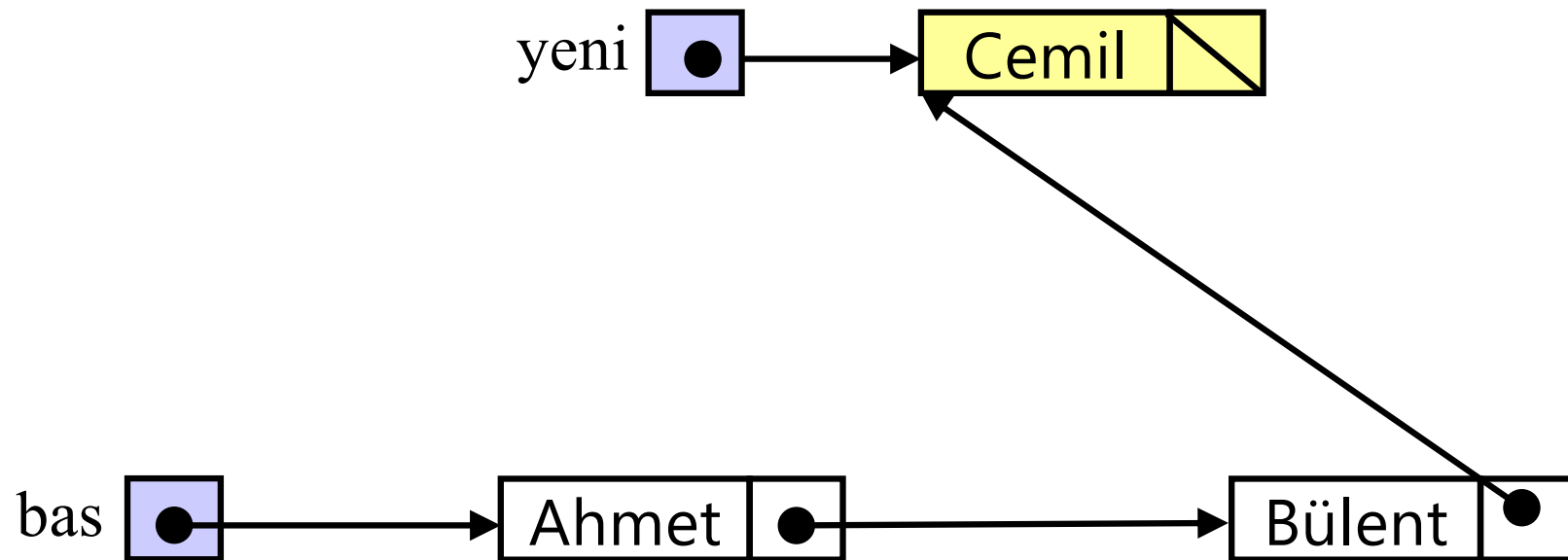
Listenin sonuna ekle (1)



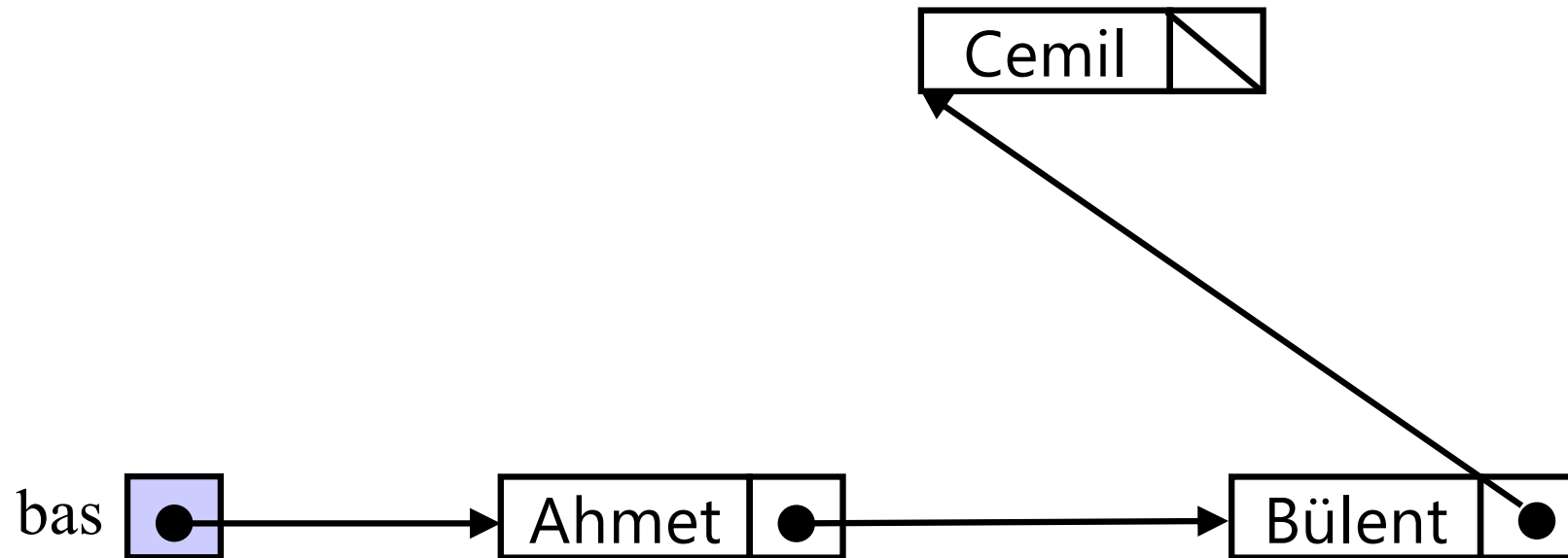
Listenin sonuna ekle (2)



Listenin sonuna ekle (3)



Listenin sonuna ekle (4)

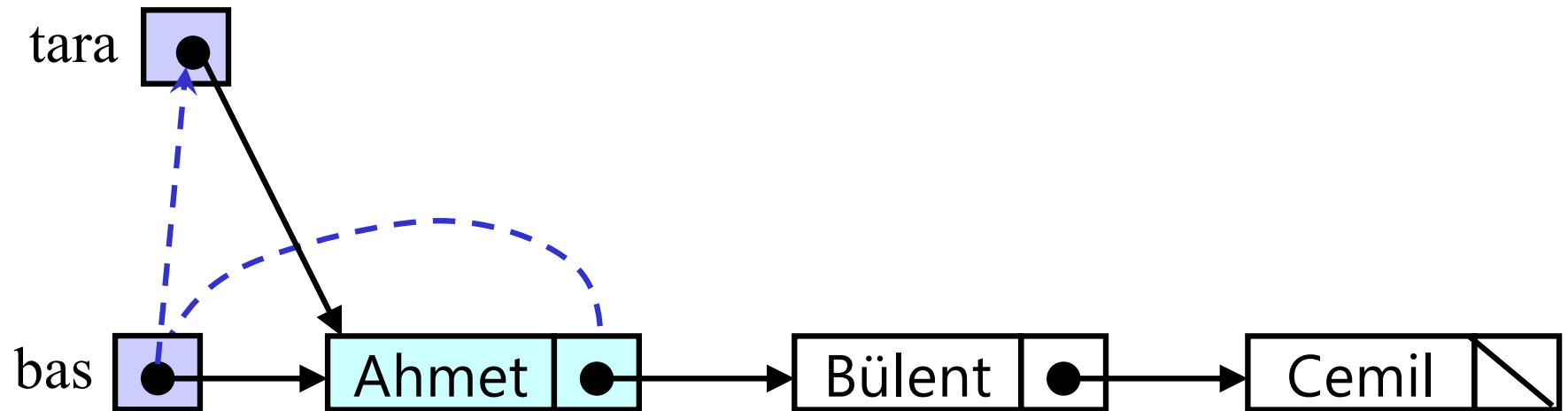


sil() Fonksiyonu

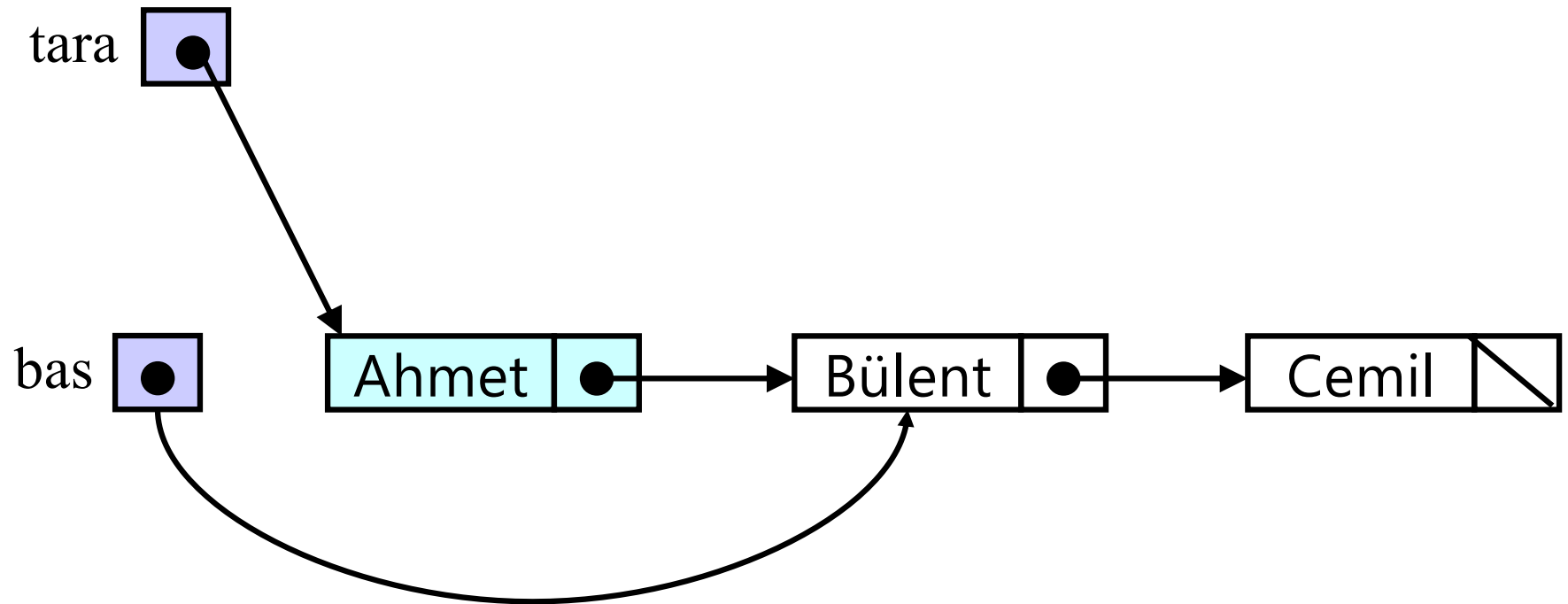
- Durum1 : Listenin başından sil
- Durum2 : Aradan sil
- Durum3 : Listenin sonundan sil
- Durum4 : Listedeki tek ve son elemanı sil

Durum1 : Listenin başından sil

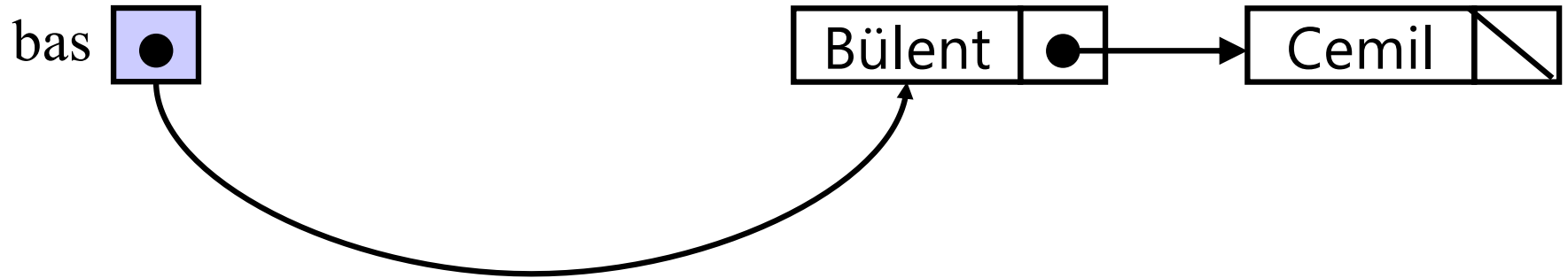
Listenin başından sil (1)



Listenin başından sil (2)

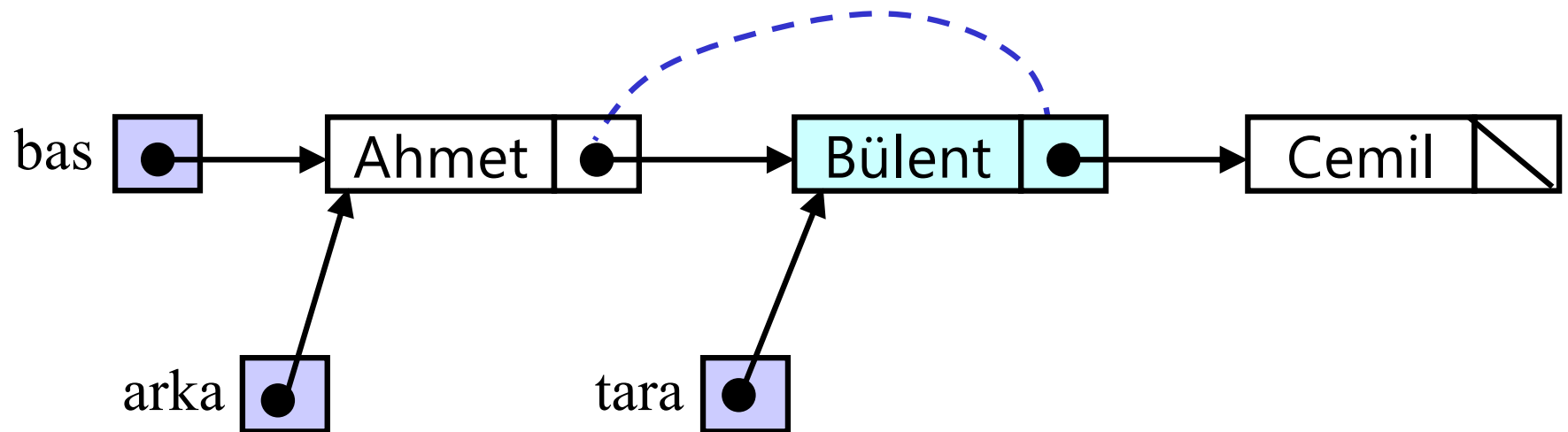


Listenin başından sil (3)

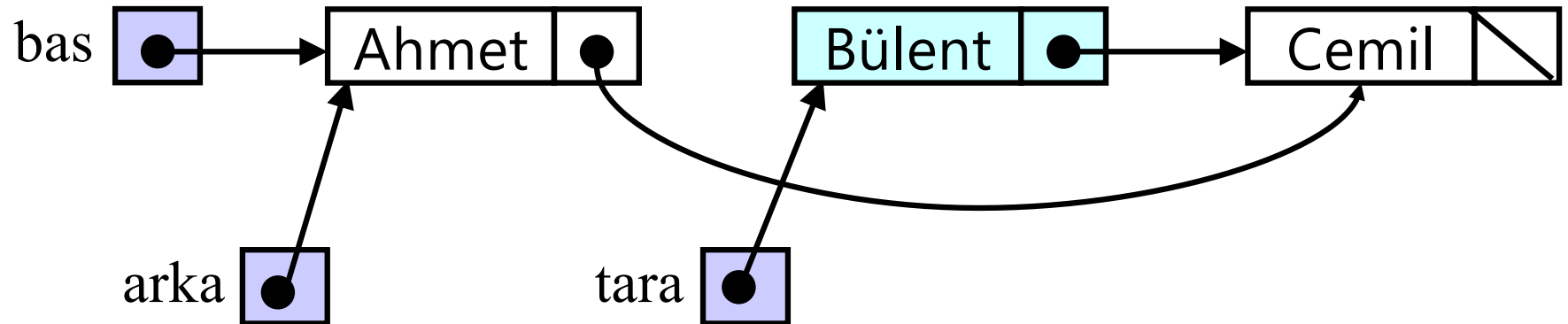


Durum2 : Aradan sil

Aradan sil (1)



Aradan sil (2)

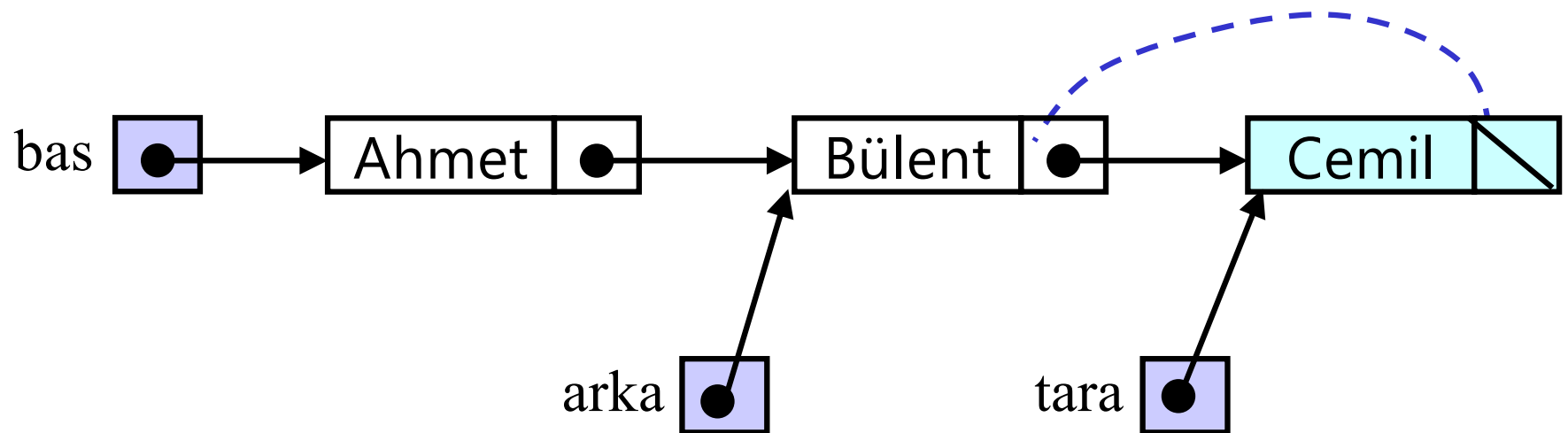


Aradan sil (3)

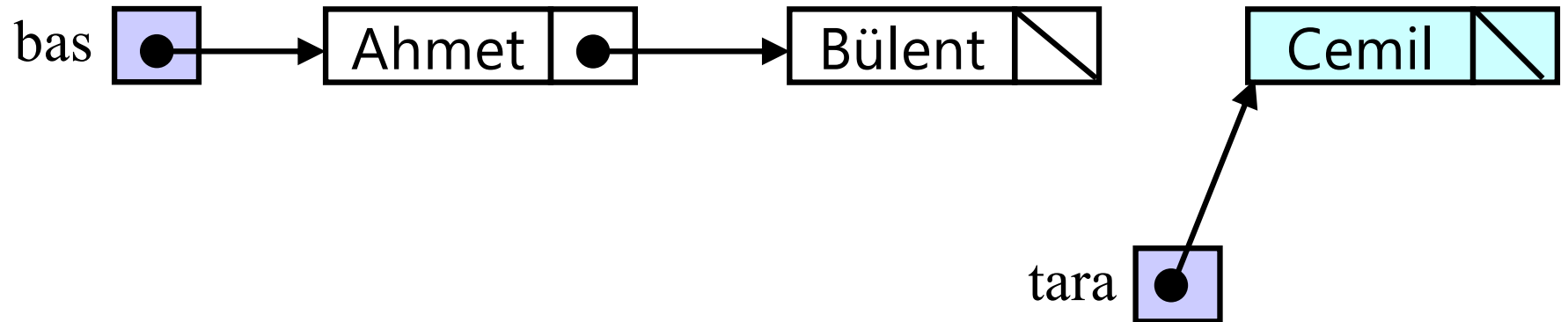


Durum3 : Listenin sonundan sil

Listenin sonundan sil (1)



Listenin sonundan sil (2)

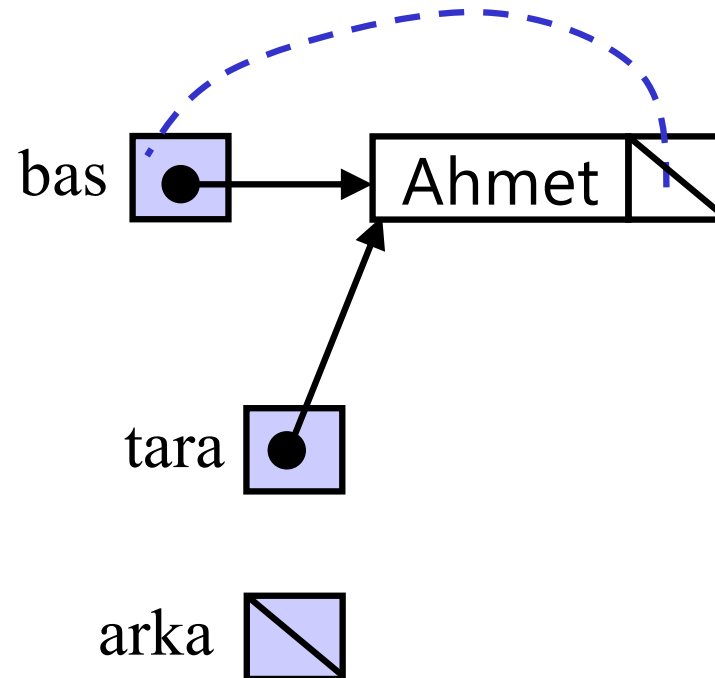


Listenin sonundan sil (3)

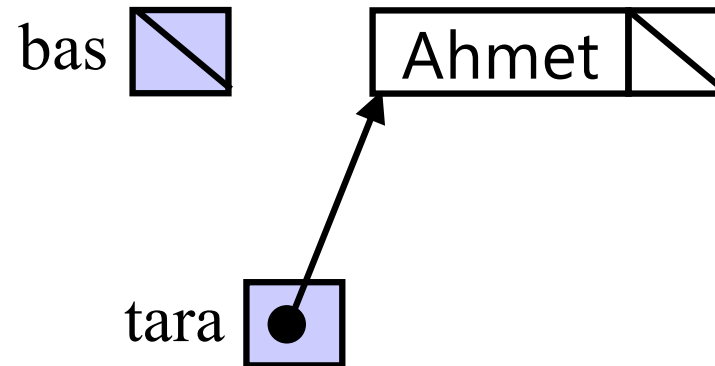


Durum4 : Listedeki tek ve
son elemanı sil

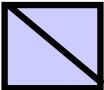
Listedeki tek ve son elemanı sil (1)



Listedeki tek ve son elemanı sil (2)



Listedeki tek ve son elemanı sil (3)

bas 

Singly Linked List Java Ugulaması

Düğüm Yapısı

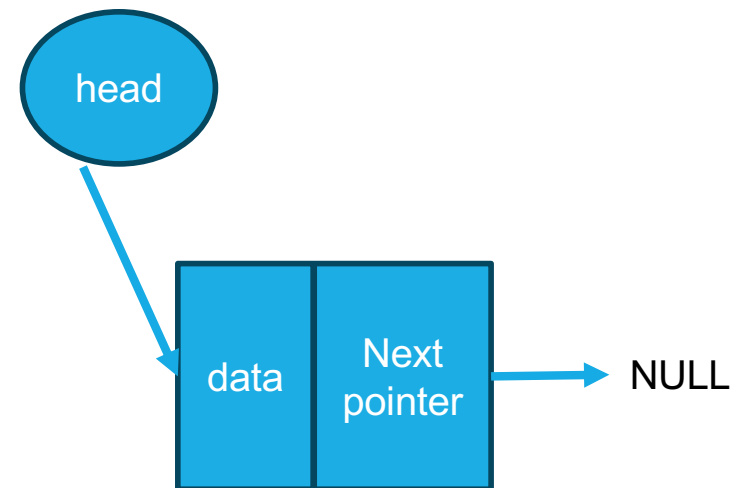
```
public class Node
{
    int data;
    public Node next;
}
```



Singly Linked List Java Uygulaması

Listenin uygulanması

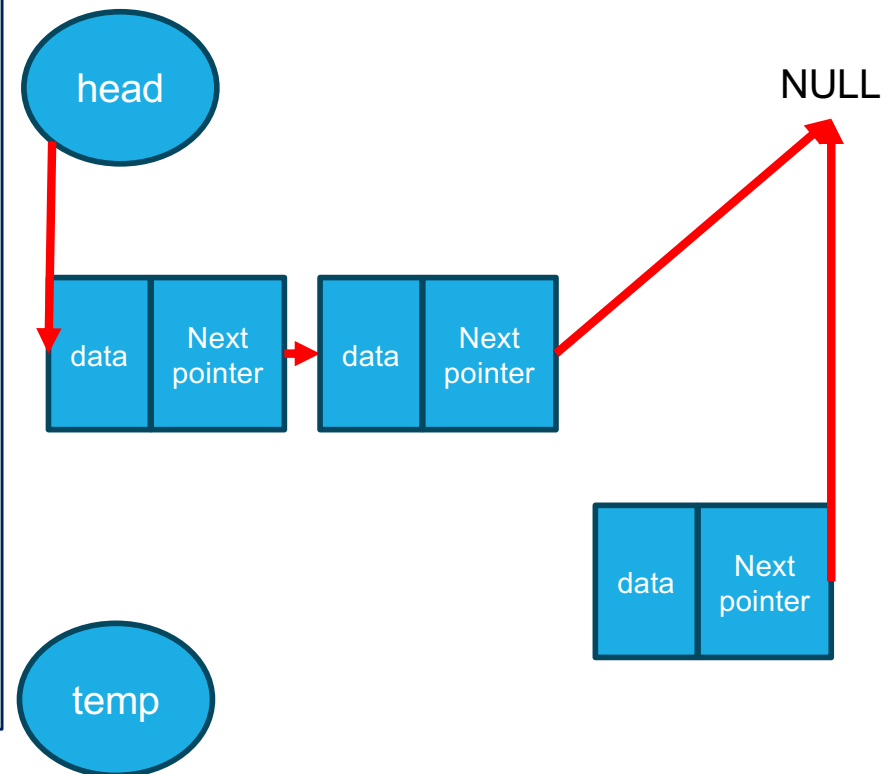
```
public class BListe {  
    Node head = null;  
    public void initialize(int data)  
    {  
        head = new Node();  
        head.data = data;  
        head.next = null;  
    }  
}
```



Singly Linked List Java Ugulaması

Listenin Sonuna Eleman Ekleme

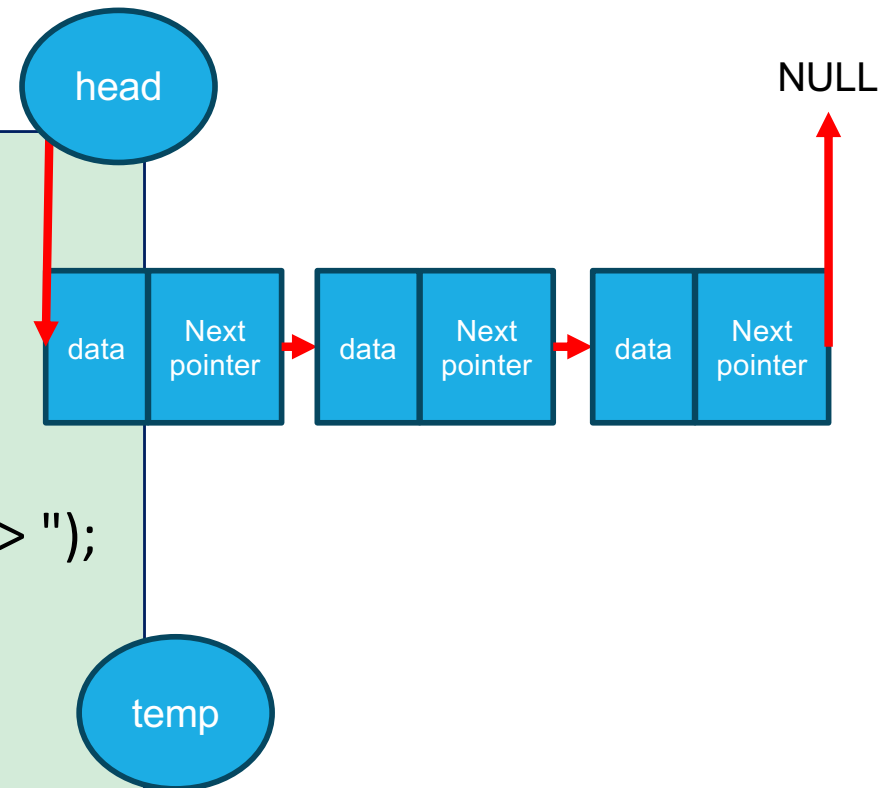
```
public void addElementLast(int data)
{
    Node newNode = new Node();
    newNode.data = data;
    newNode.next = null;
    Node temp = head;
    while(temp.next != null)
    {
        temp = temp.next;
    }
    temp.next = newNode;
}
```



Singly Linked List Java Ugulaması

Listeyi Ekrana Yazmak

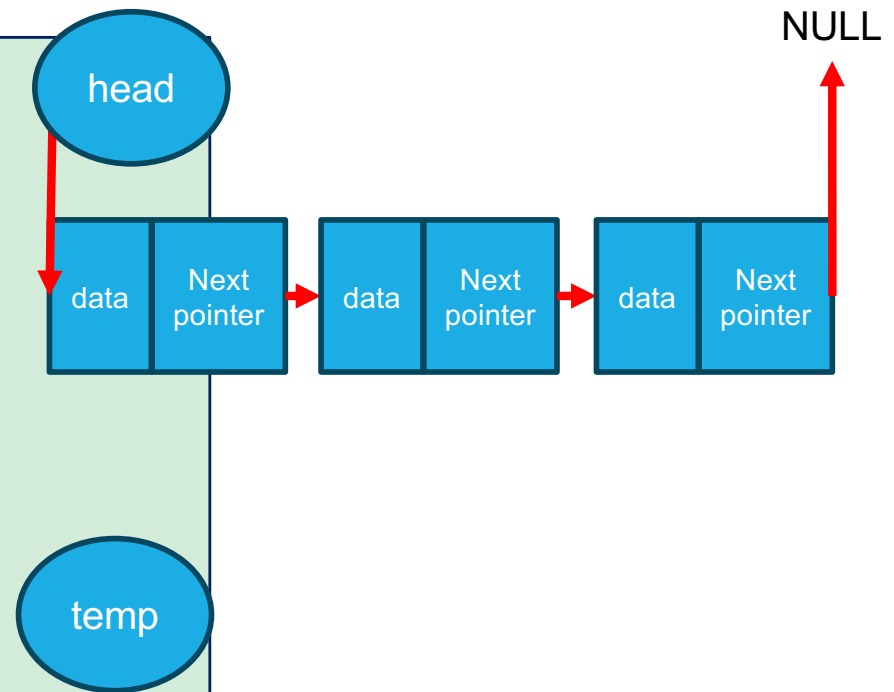
```
public void printLinkedList()
{
    Node temp = head;
    while(temp != null)
    {
        System.out.print(temp.data + " => ");
        temp = temp.next;
    }
    System.out.println("Null");
}
```



Singly Linked List Java Ugulaması

Listedeki eleman sayısını bulmak

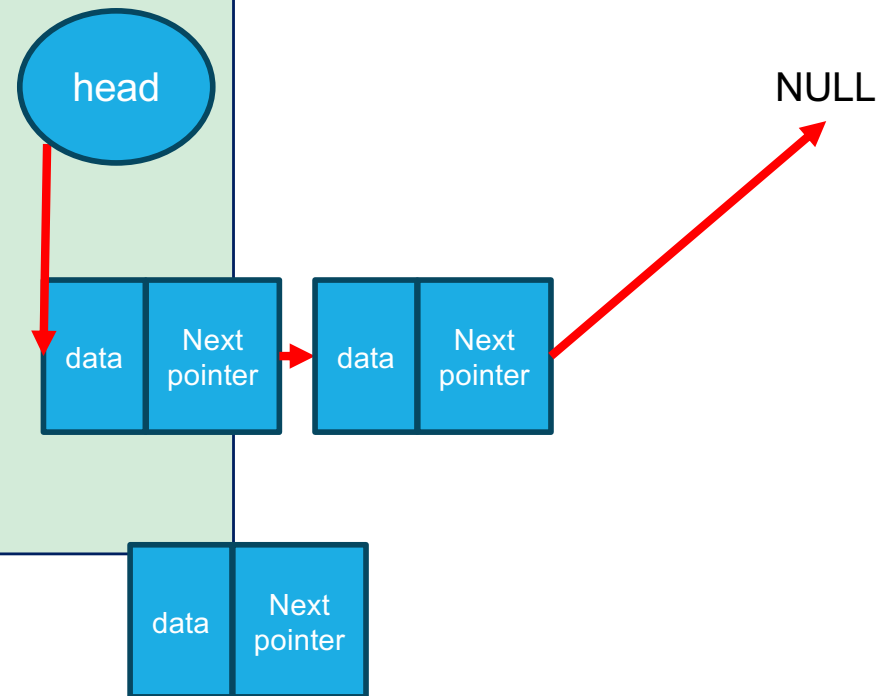
```
public int elementCount()  
{  
    Node temp = head;  
    int count = 0;  
    while(temp != null)  
    {  
        count++;  
        temp = temp.next;  
    }  
    return count;  
}
```



Singly Linked List Java Ugulaması

Listenin başına eleman eklemek

```
public void addElementFront(int data)
{
    Node newNode = new Node();
    newNode.data = data;
    newNode.next = head;
    head = newNode;
}
```

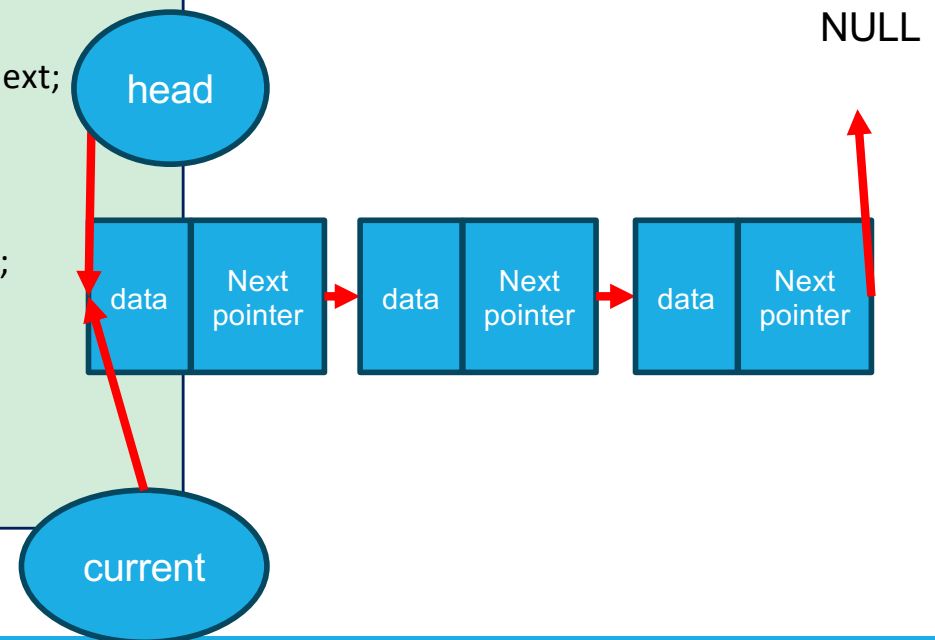


Singly Linked List Java Ugulaması

Listeden istenen bir elemanı silmek

```
public boolean
deleteNode(int data)
{
    boolean durum = false;
    Node current = head;
    Node tmp = null;
    if (current.data == data)
    {
        tmp = current.next;
        head = tmp;
    }
    else
```

```
{ // else
    {
        while(current.next != null)
        {
            if (current.next.data == data)
            {
                tmp = current.next;
                current.next = tmp.next;
                durum = true;
                break;
            }
            current = current.next;
        }
    }
    return durum;
}
```



Singly Linked List Java Ugulaması

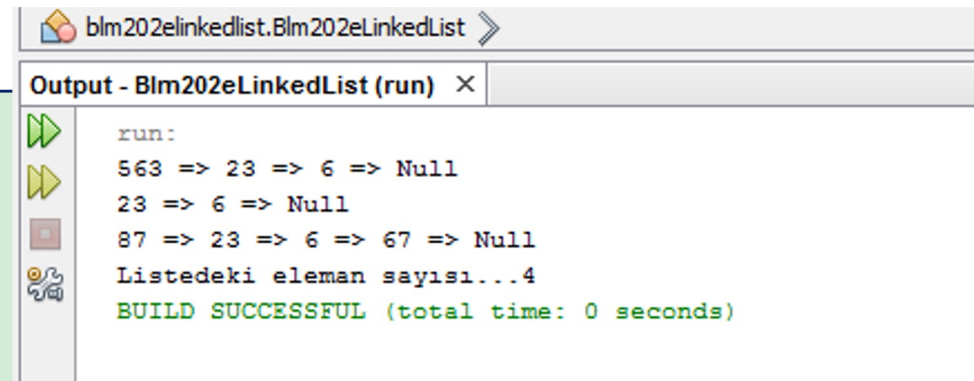
Örnek:

```
public class Blm202eLinkedList {
    public static void main(String[] args)
    {
        BListe bl = new BListe();
        bl.initialize(23);
        bl.addElementLast(6);
        bl.addElementFront(563);
        bl.printLinkedList();
        bl.deleteNode(563);
        bl.printLinkedList();
        bl.addElementLast(67);
        bl.addElementFront(87);
        bl.printLinkedList();
        System.out.println("Listedeki eleman sayısı..." + bl.elementCount());
    }
}
```

Singly Linked List Java Ugulaması

Örnek:

```
public class Blm202eLinkedList {  
    public static void main(String[] args)  
    {  
        BListe bl = new BListe();  
        bl.initialize(23);  
        bl.addElementLast(6);  
        bl.addElementFront(563);  
        bl.printLinkedList();  
        bl.deleteNode(563);  
        bl.printLinkedList();  
        bl.addElementLast(67);  
        bl.addElementFront(87);  
        bl.printLinkedList();  
        System.out.println("Listedeki eleman sayısı..." + bl.elementCount());  
    }  
}
```

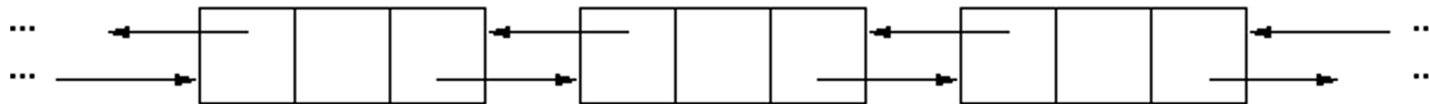


```
blm202elinkedlist.Blm202eLinkedList >  
Output - Blm202eLinkedList (run) X  
run:  
563 => 23 => 6 => Null  
23 => 6 => Null  
87 => 23 => 6 => 67 => Null  
Listedeki eleman sayısı...4  
BUILD SUCCESSFUL (total time: 0 seconds)
```

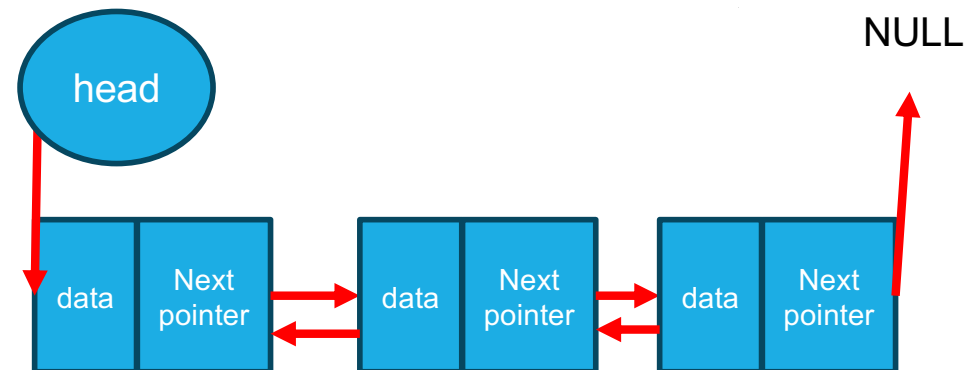
Doubly Linked List Java Ugulaması

Düğüm Yapısı

A Doubly-Linked List



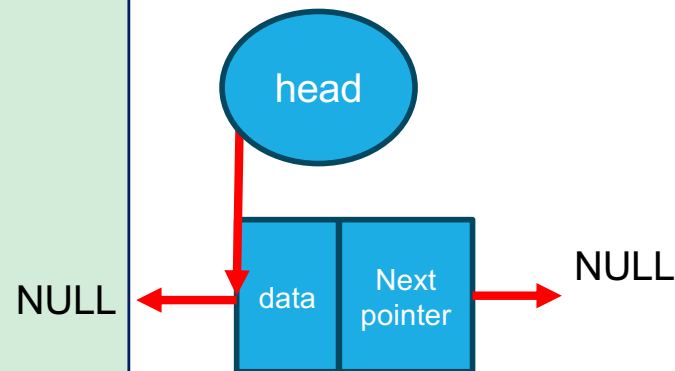
```
public class Node
{
    int data;
    Node next;
    Node prev;
}
```



Doubly Linked List Java Ugulaması

Uygulanması

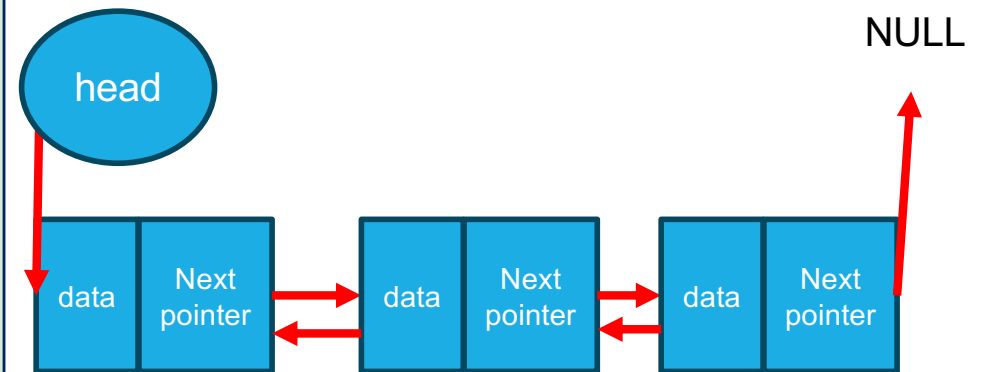
```
public class Double_Linked_List {  
    Node head = null;  
  
    public void initialize(int data)  
    {  
        head = new Node();  
        head.prev = null;  
        head.data = data;  
        head.next = null;  
    }  
}
```



Doubly Linked List Java Ugulaması

Listenin başına elaman ekleme

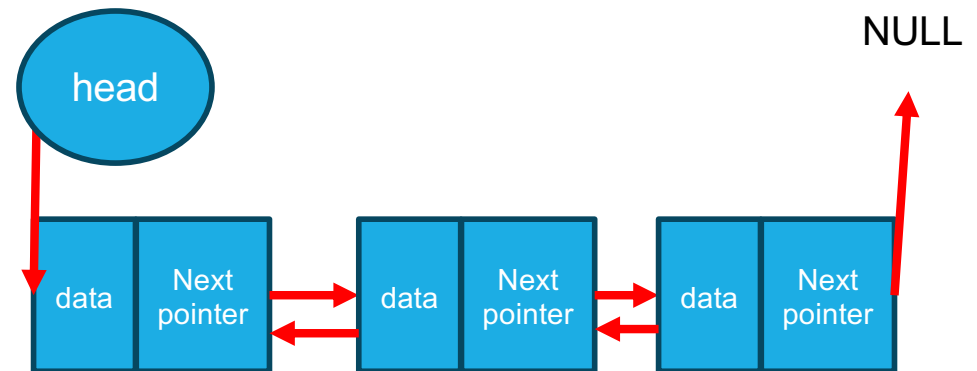
```
public void addBegin(int data)
{
    Node temp = new Node();
    temp.prev = null;
    temp.data = data;
    temp.next = head;
    head.prev = temp;
    head = temp;
}
```



Doubly Linked List Java Ugulaması

Listenin sonuna eleman ekleme

```
public void addLast(int data)
{
    Node newNode = new Node();
    Node current = new Node();
    newNode.data = data;
    newNode.next = null;
    current = head;
    while(current.next != null)
    {
        current = current.next;
    }
    current.next = newNode;
    newNode.prev = current;
}
```

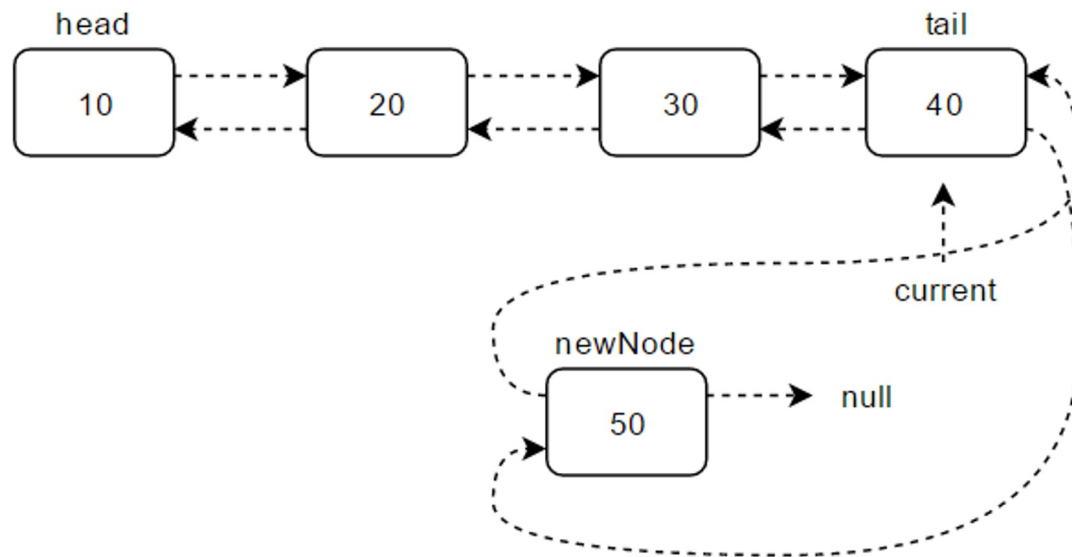


Doubly Linked List Java Ugulaması

Listeyi ekrana yazdırma

```
public void printList()
{
    if (head == null)
    {
        System.out.println("Listede eleman bulunmaktadır....");
        return;
    }
    Node temp = new Node();
    temp = head;
    while(temp !=null)
    {
        System.out.print(temp.data+ "->");
        temp = temp.next;
    }
    System.out.println("Null");
}
```

Doubly Linked List Add Last Operation



```
// sona eleman ekleme...
```

```
void addLast(T data){
```

```
    Node<T> newNode = new Node<>(data);
```

```
    if(head == null){
```

```
        head = newNode;
```

```
    }
```

```
    else{
```

```
        Node<T> current = head;
```

```
        while(current.next != null){
```

```
            current = current.next;
```

```
        }
```

```
        current.next = newNode;
```

```
        newNode.prev = current;
```

```
        newNode.next = null;
```

```
    }
```

```
}
```